

A Dissertation on
FinTwinAI: An Explainable AI-Driven Personal Financial Digital Twin
Using GraphSAGE, Probability Calibration, Macroeconomic Stress Testing

*submitted in partial fulfilment of the requirement
for the award of the degree of*

MASTER OF SCIENCE

in

Computer Science (BIG DATA ANALYTICS)

submitted by

PRITI PRAKASH MOHANTA

Enrollment No.: 2025MSBDA015

under the guidance of

Dr. Bhavna Saini

Supervisor

Department of Data Science and Analytics

Central University of Rajasthan



Department of Data Science and Analytics

School of Mathematics, Statistics and Computational Science

CENTRAL UNIVERSITY OF RAJASTHAN

Project Duration: 15 June 2026 – 15 July 2026

DECLARATION

I hereby certify that

- a. The work presented in this dissertation entitled “FinTwinAI: An Explainable AI-Driven Personal Financial Digital Twin Using GraphSAGE, Probability Calibration, Macroeconomic Stress Testing” was carried out by me under the supervision of Dr. Bhavna Saini.
- b. The work is submitted in partial fulfilment of the requirements for the degree of M.Sc. Computer Science (Big Data Analytics) at the Central University of Rajasthan.
- c. The work has not been submitted in part or in full to any other University or Institution for any degree or diploma.
- d. I further declare that the analysis, implementation and documentation were prepared for academic purposes. External research material, software libraries and methodological ideas have been acknowledged in the references. The system is a research prototype and is not a certified credit-scoring system, financial advisory system, or automated loan-approval mechanism.
- e. I have used the project notebook, dashboard source code and the dissertation format supplied for this work as the primary project sources. Where external research papers are used to explain methods or provide research context, the sources are cited in IEEE style.

Date: 24/09/2026

Name : Priti Prakash Mohanta

Place: Central University of Rajasthan, Ajmer

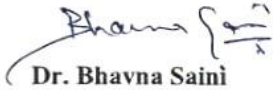
Enrollment No.: 2025MSBDA015

DEPARTMENT OF DATA SCIENCE AND ANALYTICS
CENTRAL UNIVERSITY OF RAJASTHAN, INDIA

Date: 24/09/2026

CERTIFICATE

This is to certify that the project titled “**FinTwinAI: An Explainable AI-Driven Personal Financial Digital Twin Using GraphSAGE, Probability Calibration, Macroeconomic Stress Testing**” is a record of the bonafide work done by **Priti Prakash Mohanta** (2025MSBDA015) submitted in partial fulfilment of the requirements for the award of the Degree of Master of Science (M.Sc.) in Computer Science (Big Data Analytics) of Central University of Rajasthan, from 15 June 2026 to 15 July 2026.



Dr. Bhavna Saini

Department of Data Science and Analytics
Central University of Rajasthan

Dr. Nistha Kesswani

HOD, Department of Data Science and Analytics
Central University of Rajasthan



राजस्थान केन्द्रीय विश्वविद्यालय
Central University of Rajasthan
Category-I University Status by UGC, New Delhi



Date: September 24, 2026

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Priti Prakash Mohanta (2025MSBDA015), a student of M.Sc. Computer Science (Big Data Analytics) in the Department of Data Science and Analytics, Central University of Rajasthan, has successfully completed a Research Internship under my supervision from June 15, 2026 to July 15, 2026.

Project Title: *FinTwinAI: An Explainable AI-Driven Personal Financial Digital Twin Using GraphSAGE, Probability Calibration, Macroeconomic Stress Testing*

During the internship, Mr. Priti Prakash Mohanta demonstrated sincerity, dedication, and strong technical aptitude while working on the above-mentioned project. He actively contributed to the literature review, algorithm development, implementation, experimentation, and analysis of results. His work reflected a sound understanding of the subject and an ability to effectively apply research concepts and techniques.

I wish him every success in his future endeavors.

Best Wishes,

Dr. Bhavna Saini
Assistant Professor
Department of Data Science and Analytics
Central University of Rajasthan

ACKNOWLEDGEMENTS

I place on record my deep sense of gratitude and respect to **Prof. Mamta Rani**, Dean, School of Mathematics, Statistics, Computational Science, Central University of Rajasthan, for providing the academic environment, resources, and facilities required to complete the dissertation.

I am thankful to **Dr. Nishtha Kesswani**, Head of the Department of Data Science and Analytics, and to the faculty members of the Central University of Rajasthan for providing an academic environment in which this work could be developed.

I express my sincere gratitude to **Dr. Bhavna Saini**, my project supervisor, for her guidance, academic support and encouragement throughout the project. Her suggestions helped me connect machine learning, graph neural networks, financial risk analysis, probability calibration, macroeconomic stress testing and practical dashboard development into one research-oriented prototype.

I also acknowledge the open-source communities behind Python, pandas, NumPy, scikit-learn, PyTorch, PyTorch Geometric, SHAP, Matplotlib, Plotly and Streamlit. These tools made it possible to build and evaluate the prototype within the project period. [4], [5], [6], [12]–[13].

Finally, I thank my classmates, friends and family for their support and motivation during the implementation, testing and documentation stages of FinTwinAI.

ABSTRACT

- Personal financial risk assessment is an important problem in digital lending because a useful system should estimate default probability, communicate uncertainty, explain important variables and show how risk may affect a portfolio. FinTwinAI is an academic prototype designed to combine borrower-level risk prediction, graph-based learning, explainability, probability calibration, macroeconomic stress testing and an interactive web dashboard. The term “digital twin” is used here for a data-driven financial profile rather than a complete real-time simulation of an individual.
- The project uses a large LendingClub-derived loan dataset. The notebook reports 2,260,701 records in the complete dataset. Binary modelling focuses on Fully Paid and Charged Off/Default outcomes. A temporal split is used: training data through issue year 2015, validation data from 2016, and testing data from 2017 onward. Twenty-six financially meaningful features are prepared for the graph model. A K-nearest-neighbour similarity graph is then constructed from a 30,000-node sample. The final graph contains 369,098 undirected edge entries.
- Two neural models are evaluated: a multilayer perceptron (MLP) baseline and a three-layer GraphSAGE model. The MLP achieves 74.30% accuracy, 9.77% default recall and 0.5533 ROC-AUC. GraphSAGE achieves 64.32% accuracy, 61.13% default recall, 42.19% F1-score, 0.6858 ROC-AUC and 0.3560 average precision. Platt-style probability calibration reduces the Brier score from 0.222188 to 0.158626 for GraphSAGE and from 0.228459 to 0.167802 for MLP.
- The digital-twin layer converts model probabilities into a financial health score, risk category, exposure and expected loss, and connects each profile to a macroeconomic stress index. The final dashboard dataset contains 5,000 profiles, total exposure of 72,858,850 in the original dataset units and baseline expected loss of 35,861,092.02. Hypothetical moderate and severe stress scenarios raise expected loss to 42,763,119.00 and 51,566,807.22 respectively. The project is implemented with Python, Jupyter Notebook, pandas, NumPy, scikit-learn, PyTorch, PyTorch Geometric, SHAP, Matplotlib, Plotly and Streamlit. The system is intended for academic research and education, not real-world lending decisions.

LIST OF TABLES

Table No.	Table Title	Page No.
3.1	Dataset and temporal split summary	8
3.2	GraphSAGE feature set	9
3.3	Graph construction statistics	11
3.4	GraphSAGE configuration	11
3.5	MLP baseline configuration	12
4.1	Software and libraries	16
4.2	Dashboard modules	17
4.3	Risk-category thresholds	20
5.1	Final model performance comparison	22
5.2	Additional classification metrics	23
5.3	Probability calibration results	26
5.4	Calibrated probability summary	27
5.5	Digital-twin portfolio summary	28
5.6	Stress scenario analysis	32
5.7	Methodology and leakage audit	36
A.1	Core mathematical formulae	42
B.1	Full monthly portfolio table	42
C.1	Dashboard page inventory	43
E.1	Key numerical results at a glance	44

LIST OF FIGURES

Figure No.	Figure Title	Page No.
3.1	FinTwinAI end-to-end workflow	7
3.2	Loan status distribution in the exploratory data analysis	7
3.3	Distribution of loans by issue year	8
3.4	Default-rate trend by issue year	9
3.5	GraphSAGE training and validation loss	12
3.6	GraphSAGE threshold performance analysis	13
4.1	FinTwinAI dashboard landing page and loaded project files	17
4.2	Income and Loan Details input interface	17
4.3	Loan Repayment Summary generated by FinTwinAI	19
4.4	Explanation in Simple Language and educational health interpretation	19
4.5	Important Financial Indicators and personalized recommendations	20
5.1	Final MLP versus GraphSAGE performance comparison	22
5.2	GraphSAGE test confusion matrix	23
5.3	GraphSAGE ROC curve	24
5.4	GraphSAGE precision-recall curve	25
5.5	ROC comparison between MLP and GraphSAGE	25
5.6	Probability calibration comparison	26
5.7	GraphSAGE reliability diagram	27
5.8	Borrower distribution by risk category	28
5.9	Exposure and expected loss by risk category	29
5.10A	FRED unemployment-rate series used in the macroeconomic module	29
5.10B	FRED CPI series used in the macroeconomic module	30
5.10C	FRED federal-funds-rate series used in the macroeconomic module	30
5.10D	FRED GDP series used in the macroeconomic module	31
5.11	Illustrative Macroeconomic Stress Index	31
5.12	Expected loss under baseline, moderate and severe stress	32
5.13	Monthly baseline expected-loss trend	33
5.14	Monthly macroeconomic stress index	33
5.15A	Top exported SHAP feature importance values	34
5.15B	GraphSAGE input-gradient feature importance	34
5.16	Illustrative survival curves used by the dashboard	35

CONTENTS

Acknowledgements	iv
Abstract	v
List of Tables	vi
List of Figures	vii
CHAPTER 1: INTRODUCTION	1
1.1 Introduction and Motivation	1
1.2 Research Problem	1
1.3 Research Questions	1
1.4 Objectives	1
1.5 Organization of the Report	2
CHAPTER 2: BACKGROUND MATERIAL	3
2.1 Conceptual Overview	3
2.2 Credit Risk and Probability of Default	3
2.3 Feature Similarity and Graph Representation	3
2.4 GraphSAGE	3
2.5 Multilayer Perceptron Baseline	4
2.6 Explainable AI and SHAP	4
2.7 Probability Calibration	4
2.8 Expected Loss	5
2.9 Macroeconomic Stress Testing	5
2.10 Survival Analysis	5
2.11 Research Context and Literature	5
2.12 Research Gap Addressed by FinTwinAI	6
CHAPTER 3: METHODOLOGY	7
3.1 End-to-End Methodology	7
3.2 Dataset Preparation	7
3.3 Temporal Split	8
3.4 Feature Engineering	9
3.5 Leakage Control	10

3.6	Missing Values and Standardization	10
3.7	KNN Graph Construction	11
3.8	GraphSAGE Model	11
3.9	MLP Baseline	12
3.10	Threshold Selection	13
3.11	Probability Calibration	14
3.12	System Workflow	14
CHAPTER 4:	IMPLEMENTATION	16
4.1	Software Environment	16
4.2	Data-to-Dashboard Output Files	16
4.3	Dashboard Architecture	16
4.3.1	Dashboard Landing Page and File Loading	16
4.3.2	Income and Loan Details Interface	17
4.4	Personal Financial Health Module	18
4.4.1	Loan Repayment Summary	18
4.4.2	Explanation in Simple Language	19
4.4.3	Financial Indicators and Recommendations	19
4.5	Digital-Twin Profile	20
4.6	Risk Categories	20
4.7	Explainability Implementation	20
4.8	Survival Module	21
4.9	Responsible-Use Controls	21
CHAPTER 5:	RESULTS AND ANALYSIS	22
5.1	Final Model Performance	22
5.2	Additional Classification Metrics	23
5.3	GraphSAGE Confusion Matrix	23
5.4	ROC and Precision-Recall Analysis	24
5.5	Probability Calibration	26
5.6	Calibrated Probability Distribution	27
5.7	Digital-Twin Portfolio Results	28
5.8	Macroeconomic Indicators and Stress Index	29
5.9	Stress Scenario Analysis	32
5.10	Monthly Portfolio Results	33

5.11	SHAP and Feature Importance	34
5.12	Survival Analysis	35
5.13	Methodology Audit	36
CHAPTER 6: CONCLUSIONS AND FUTURE SCOPE		37
6.1	Conclusions	37
6.2	Main Contributions	37
6.3	Key Limitations	38
6.4	Future Work	38
6.5	Final Statement	38
References		40
Annexures		42

CHAPTER 1: INTRODUCTION

1.1 Introduction and Motivation

Financial risk systems traditionally focus on producing a score or a binary decision. For a research-oriented analytics system, that is not enough. A useful prototype should show what the model predicts, how reliable the probability is, what variables contribute to the prediction, how risk is distributed across a portfolio, and how the result changes under hypothetical stress.

FinTwinAI addresses this wider requirement. It treats each loan record as a compact digital representation of a financial profile. The profile contains a predicted probability of default, a health score, a risk category, exposure, expected loss and macroeconomic context. This interpretation is deliberately limited: the digital twin is a data-driven analytical profile, not a complete real-time replica of a person, household or financial institution.

The project also investigates graph learning. A loan does not exist in isolation in a real financial system. Similar borrowers may share patterns in income, debt, credit history and loan structure. Because the supplied dataset does not contain confirmed borrower-to-borrower relationships, FinTwinAI creates a feature-similarity graph using K-nearest neighbours. This makes the graph a research device for contextual learning rather than evidence of real financial contagion.

1.2 Research Problem

The central research problem is to design an explainable financial-risk prototype that can combine tabular machine learning, graph neural networks, probability calibration, portfolio expected-loss analysis, macroeconomic sensitivity analysis and a usable web interface. The challenge is not only predictive accuracy. The project must also address class imbalance, temporal leakage, probability reliability, interpretability and responsible use.

1.3 Research Questions

1. Can graph-based neighbourhood aggregation identify default cases differently from an MLP that uses the same node features but ignores graph edges?
2. Does post-hoc probability calibration improve the reliability of the model's probability estimates?
3. Can model outputs be translated into a digital-twin profile containing health, exposure and expected-loss indicators?
4. Can macroeconomic stress scenarios provide an understandable sensitivity analysis for portfolio risk?
5. Can a dashboard present these results in a form that is easy to inspect without reading notebook code?

1.4 Objectives

- Prepare a modelling dataset while excluding variables that may leak target information.
- Engineer financially meaningful variables such as average FICO, loan-to-income and installment-to-income ratios.
- Construct a KNN feature-similarity graph from 26 selected variables.
- Train a GraphSAGE classifier and compare it with an MLP baseline.
- Evaluate accuracy, precision, recall, F1-score, ROC-AUC, average precision and confusion matrices.
- Calibrate model probabilities using a logistic/Platt-style post-processing step.

- Create digital-twin profiles with probability of default, health score, risk category, exposure and expected loss.
- Introduce macroeconomic indicators and hypothetical stress multipliers.
- Build an interactive Streamlit dashboard for executive, portfolio, network, explainability and stress views.
- Document limitations and future work required before any operational use.

1.5 Organization of the Report

Chapter 1 introduces the motivation and objectives.

Chapter 2 explains the theoretical background and related research.

Chapter 3 describes data preparation, feature engineering, graph construction, model design, calibration, stress testing and validation.

Chapter 4 documents implementation and dashboard development.

Chapter 5 presents numerical results, graphs and interpretation.

Chapter 6 gives conclusions, limitations and future scope.

References and annexures provide research sources, formulas, reproducibility notes and extended numerical tables.

CHAPTER 2: BACKGROUND MATERIAL

2.1 Conceptual Overview

FinTwinAI combines credit-risk prediction, graph-based machine learning, explainable AI, probability calibration, expected-loss analysis, macroeconomic stress testing, and digital-twin visualization into one integrated framework.

The main idea is to transform borrower and loan information into a digital financial profile that can show risk, expected loss, explanations, and changes under different financial scenarios.

2.2 Credit Risk and Probability of Default

Credit-risk modelling attempts to estimate the probability that a borrower or loan will experience a defined adverse outcome. In this project the target is binary: $Y=1$ represents default and $Y=0$ represents non-default. The model estimates $P(Y=1|X)$, where X contains loan and borrower characteristics available for modelling.

Empirical work on LendingClub data supports the importance of borrower characteristics, loan structure and macroeconomic conditions in explaining default behaviour. Croux et al. analysed more than one million LendingClub loans and identified loan maturity and borrower-related characteristics among important determinants of fintech-loan default [8]. This provides a research basis for using borrower and loan attributes as the main inputs in FinTwinAI.

$$p_i = P(Y_i = 1 | X_i) \quad \dots(1)$$

A probability is not automatically a decision. It must be interpreted together with threshold choice, class imbalance, error costs, calibration and validation quality. This is especially important here because the default class is smaller than the non-default class.

2.3 Feature Similarity and Graph Representation

A graph is represented as $G=(V,E)$, where V is the set of nodes and E is the set of edges. In FinTwinAI, one node corresponds to one sampled loan. The node contains the 26 selected numerical features. An edge means that two loans are close in standardized feature space.

Graph-based representations are useful when observations can be related through a meaningful similarity structure. In this dissertation, the graph is intentionally constructed from feature similarity rather than observed borrower relationships, so the resulting neighbourhoods provide contextual information without claiming actual financial connections.

$$d(x_i, x_j) = \sqrt[\ell]{\sum_m (x_{im} - x_{jm})^2} \quad \dots(2)$$

For each node, the eight closest neighbours are selected. The graph is therefore based on similarity, not a verified relationship. The GraphSAGE paper introduced an inductive approach in which node representations are generated by sampling and aggregating neighbourhood information, allowing the learned function to generalize to unseen nodes [1].

2.4 GraphSAGE

GraphSAGE learns a function that combines a node's own representation with an aggregation of its neighbours. A simplified mean-aggregation layer is written as follows.

Hamilton, Ying and Leskovec introduced GraphSAGE as an inductive representation-learning framework that learns how to generate node embeddings by sampling and aggregating local

neighbourhood information [1]. This is relevant to FinTwinAI because the model can combine a loan's own financial characteristics with information from similar loans in the constructed graph.

$$h_v^{(k)} = \sigma(W_k \cdot \text{CONCAT} (h_v^{(k-1)}, \text{MEAN} (\{h_u^{(k-1)} : u \in N(v)\}))) \quad \dots(3)$$

In the project, a three-layer GraphSAGE model with 64 hidden channels, 0.30 dropout and two output classes is trained using Adam. The model has 11,906 trainable parameters.

2.5 Multilayer Perceptron Baseline

The MLP is used as a controlled baseline. It receives the same standardized 26-dimensional node representation but does not use the graph edge structure. Its architecture contains a 26→64 linear layer, ReLU activation, dropout, a 64→64 hidden layer, another ReLU and dropout, followed by a 64→2 output layer. The MLP contains 6,018 trainable parameters.

The baseline is important because it isolates the contribution of graph structure: both models receive the same engineered node features, while only GraphSAGE additionally uses neighbourhood information. The comparison therefore focuses on whether relational context adds information beyond a conventional feature-only neural network.

2.6 Explainable AI and SHAP

SHAP (SHapley Additive exPlanations) provides a framework for assigning contribution values to features. Lundberg and Lee introduced SHAP as a unified additive explanation framework for model predictions [2]. A simplified additive representation is: [13].

Lundberg and Lee formulated SHAP as an additive explanation framework that attributes a model output to individual input features [2]. In FinTwinAI, this principle is used to move beyond a single risk probability and provide feature-level information that can help a user understand which inputs are influential in the model output.

$$f(x) \approx \varphi_0 + \sum_i \varphi_i \quad \dots(4)$$

The project includes SHAP calculations and exports feature-importance values. The final notebook also recovers the original 26 feature names. Because an intermediate export labelled the features as Feature_1 through Feature_26, the final report treats the anonymized SHAP labels cautiously and records the feature-name mapping as a reproducibility item.

2.7 Probability Calibration

A model can rank cases correctly while producing probabilities that are too high or too low. Probability calibration addresses this problem. Platt's method uses a sigmoid transformation to map model scores to probabilities [3]. In this project a logistic calibrator is fitted on validation-set log-odds and then applied to the test probabilities.

Platt's work demonstrated that classifier scores can be transformed into probabilistic outputs using a sigmoid-based calibration approach [3]. For credit-risk applications, this distinction is useful because a probability of default is intended to represent a quantity that can be interpreted and aggregated, not merely a ranking score.

$$\text{Brier} = (1/n) \sum_i (p_i - y_i)^2 \quad \dots(5)$$

Lower Brier score is better. Calibration is therefore evaluated separately from discrimination metrics such as ROC-AUC.

2.8 Expected Loss

Expected loss gives a simple portfolio interpretation of probability of default and exposure. The project assumes loss-given-default of 1.0 for the prototype, so expected loss is calculated as:

The expected-loss formulation is used here as a portfolio-level interpretation of model probabilities rather than as a regulatory loss-estimation framework. This keeps the prototype transparent while making it possible to aggregate predicted risk across profiles.

$$EL_i = Exposure_i \times PD_i \quad \dots(6)$$

This is a simplified expected-loss relationship. It is not a complete regulatory expected-loss calculation because real credit risk systems may model probability of default, exposure at default and loss given default separately.

2.9 Macroeconomic Stress Testing

The project retrieves monthly macroeconomic series including unemployment rate, CPI, federal funds rate and GDP from FRED. It constructs an illustrative Macro Stress Index (MSI) from standardized macro variables. The stress engine then applies assumed probability multipliers: 1.00 for baseline, 1.20 for moderate stress and 1.50 for severe stress. [14].

The use of macroeconomic information is also supported by empirical LendingClub research. Croux et al. reported that macroeconomic variables can contribute to explaining fintech-loan default alongside borrower and contractual characteristics [8]. In addition, the FinTech literature highlights the growing role of connected data and digital technologies in financial applications [10].

$$PD_{moderate} = \min(1, 1.20 \times PD_{baseline}) \quad \dots(7)$$

$$PD_{severe} = \min(1, 1.50 \times PD_{baseline}) \quad \dots(8)$$

These multipliers are sensitivity assumptions. They should not be interpreted as a calibrated macroeconomic forecast.

2.10 Survival Analysis

Survival analysis models time until an event. A scientifically valid credit-survival model needs an origination date, event/default date, follow-up duration and censoring information. The notebook contains exploratory Kaplan–Meier/Cox-style calculations using loan term and the default indicator, but the event timing is not a verified time-to-default measurement. Therefore the dashboard's survival page is explicitly illustrative.

Cox's proportional-hazards framework provides a classical foundation for relating covariates to time-to-event outcomes [7]. The present project does not claim a fully validated credit-survival model because verified event timing and censoring information are not available; the survival component is therefore retained as an exploratory illustration.

$$S(t) = e^{(-\lambda t)} \quad \dots(9)$$

2.11 Research Context and Literature

Prior research provides a foundation for each major component of FinTwinAI. Croux et al. examined more than one million LendingClub loans and found that loan maturity, borrower characteristics and macroeconomic variables are relevant to fintech-loan default prediction [8]. Turiel and Aste studied artificial-intelligence methods for peer-to-peer loan acceptance and default prediction, showing the

value of applying multiple machine-learning approaches to lending decisions [9]. These studies motivate the use of a large loan-level dataset and a comparative modelling design in the present work.

For the graph-learning component, Hamilton, Ying and Leskovec introduced GraphSAGE as an inductive method based on neighbourhood sampling and aggregation [1]. Lundberg and Lee established SHAP as a unified framework for interpreting model predictions through feature contributions [2], while Platt's calibration method provides the basis for transforming model scores into more interpretable probabilities [3]. Research on FinTech applications further emphasizes the role of data-driven technologies in financial services [10]. Chatterjee et al. discuss digital-twin applications in financial fraud detection and highlight both opportunities and practical challenges, supporting the use of a digital representation while also reinforcing the need for validation and governance [11].

2.12 Research Gap Addressed by FinTwinAI

The reviewed studies establish strong foundations for individual components of the project, but FinTwinAI combines these components into a single research prototype. Its contribution is therefore integration rather than claiming a new standalone algorithm: graph learning [1], explainability [2], probability calibration [3], fintech credit-risk evidence [8], [9], and digital-twin-oriented financial applications [10], [11] are connected with portfolio expected-loss and stress-sensitivity analysis. The project consequently provides a platform for examining not only predictive performance, but also probability reliability, interpretability, portfolio aggregation and scenario-based risk analysis.

CHAPTER 3: METHODOLOGY

3.1 End-to-End Methodology

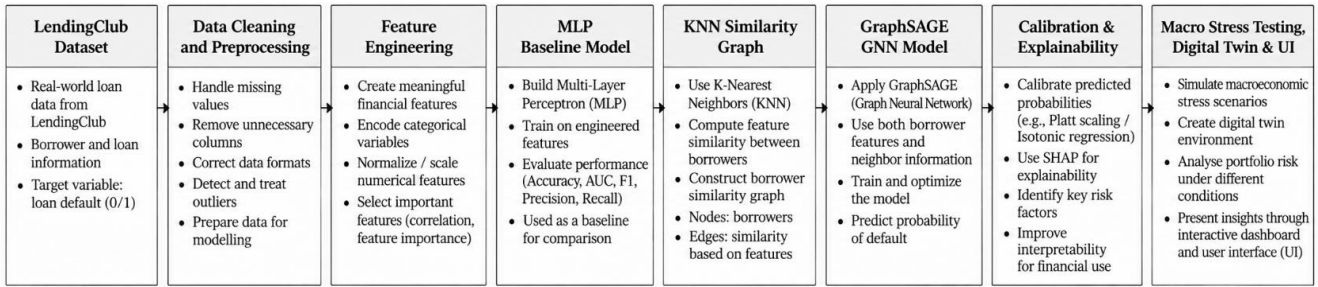


Figure 3.1. FinTwinAI end-to-end workflow

The complete workflow begins with the LendingClub-derived dataset and proceeds through cleaning, target construction, feature engineering, temporal splitting, baseline MLP modelling, KNN graph construction, GraphSAGE training, threshold analysis, probability calibration, explainability, digital-twin creation, macro stress testing and Streamlit dashboard export. [4], [5], [6] [12] [15].

3.2 Dataset Preparation

The notebook reports 2,260,701 records in the complete accepted-loan dataset. Binary modelling retains Fully Paid and Charged Off/Default outcomes. One notebook printout reports 1,345,310 usable records, while a later modelling table reports 1,345,350 rows. This 40-row discrepancy is retained explicitly rather than hidden. Before publication-quality reproducibility is claimed, the original raw file should be re-run and the final row count reconciled.

The reported class distribution is 1,076,751 non-default observations and 268,599 default observations, approximately 80.04% and 19.97%. This imbalance is the reason class-weighted cross-entropy is used during neural-network training.

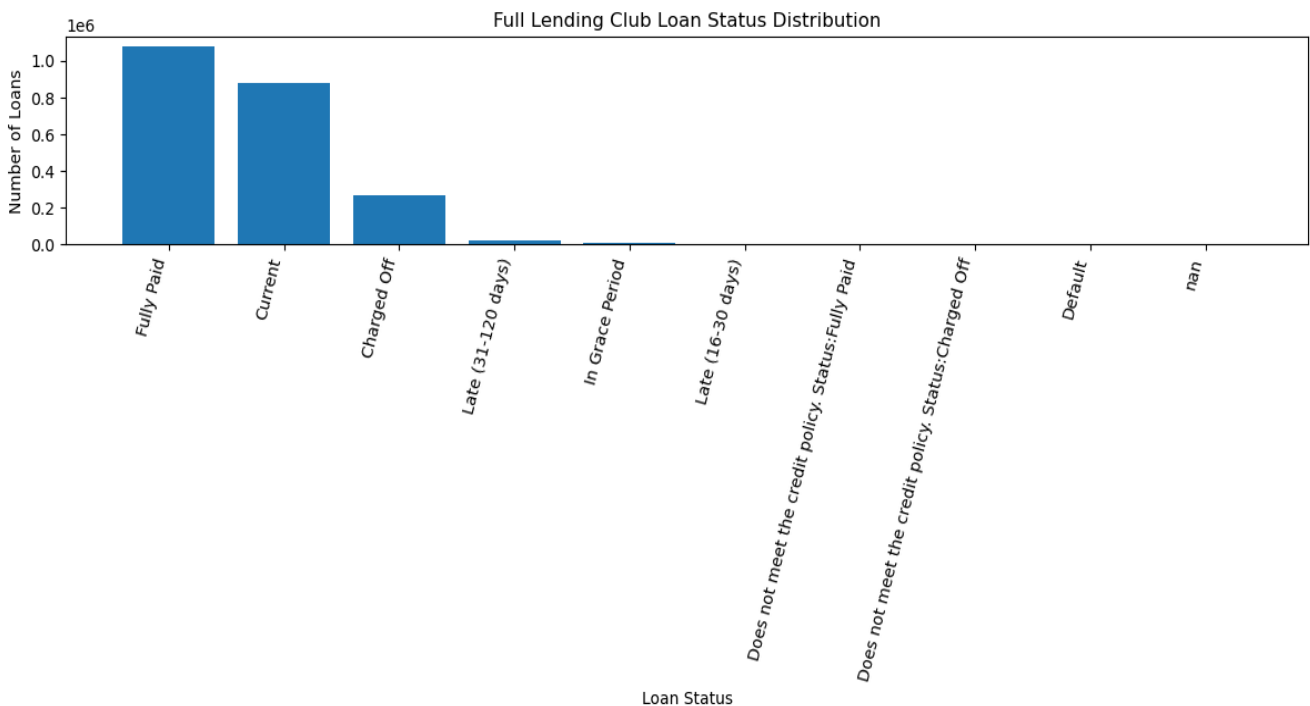


Figure 3.2. Loan status distribution in the exploratory data analysis

Figure 3.2 illustrates the class imbalance in the binary target. The reported data contain 1,076,751 non-default observations and 268,599 default observations, corresponding to approximately 80.04% and 19.97%. The figure therefore provides the visual reason for using class-weighted learning rather than relying only on overall accuracy.

3.3 Temporal Split

Table 3.1. Dataset and temporal split summary

Split	Time rule	Records
Training	Issue year ≤ 2015	826,606
Validation	Issue year = 2016	293,105
Testing	Issue year ≥ 2017	225,639

Table 3.1 shows the chronological division used for model development. The training set contains 826,606 records from issue years up to 2015, the validation set contains 293,105 records from 2016, and the test set contains 225,639 records from 2017 onward. Together these three splits give 1,345,350 modelling records, matching the later modelling-table count. The arrangement preserves time order, so later observations are not used to train the model.

The temporal design reduces look-ahead leakage and approximates a deployment setting in which a model is trained on earlier loans and tested on later loans. The graph experiment samples 20,000 training nodes, 5,000 validation nodes and 5,000 test nodes from the modelling data.

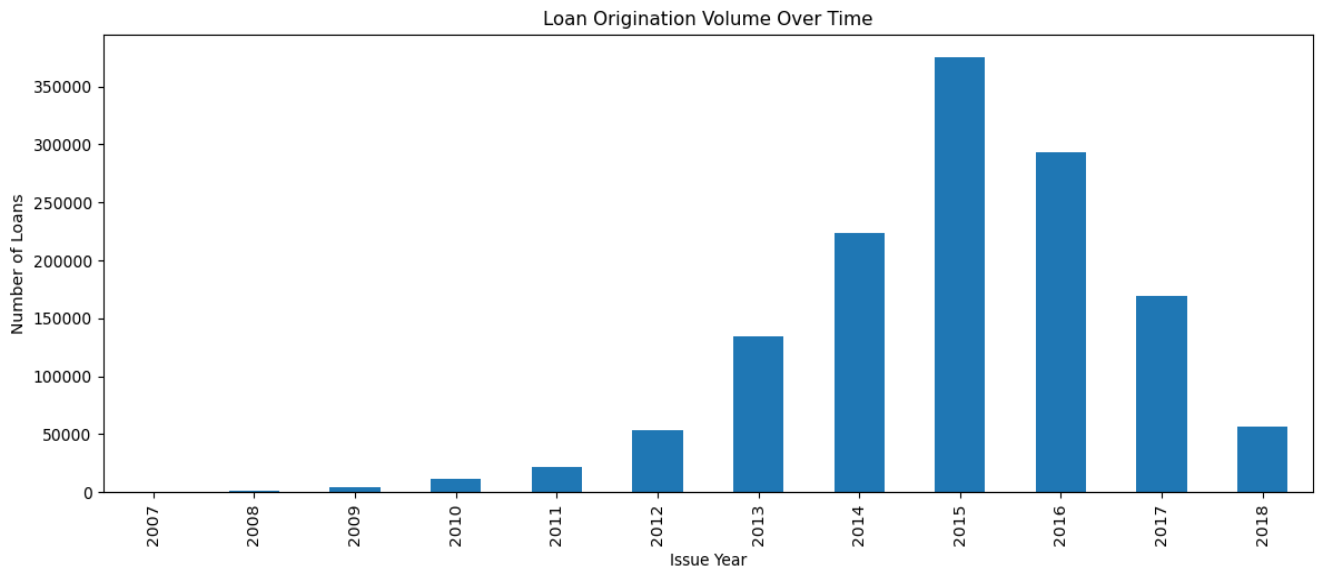


Figure 3.3. Distribution of loans by issue year

Figure 3.3 shows how the available loans are distributed across issue years. This view is important because the project uses time as part of the validation design rather than treating all observations as interchangeable. The figure provides context for the split into training through 2015, validation in 2016 and testing from 2017 onward.

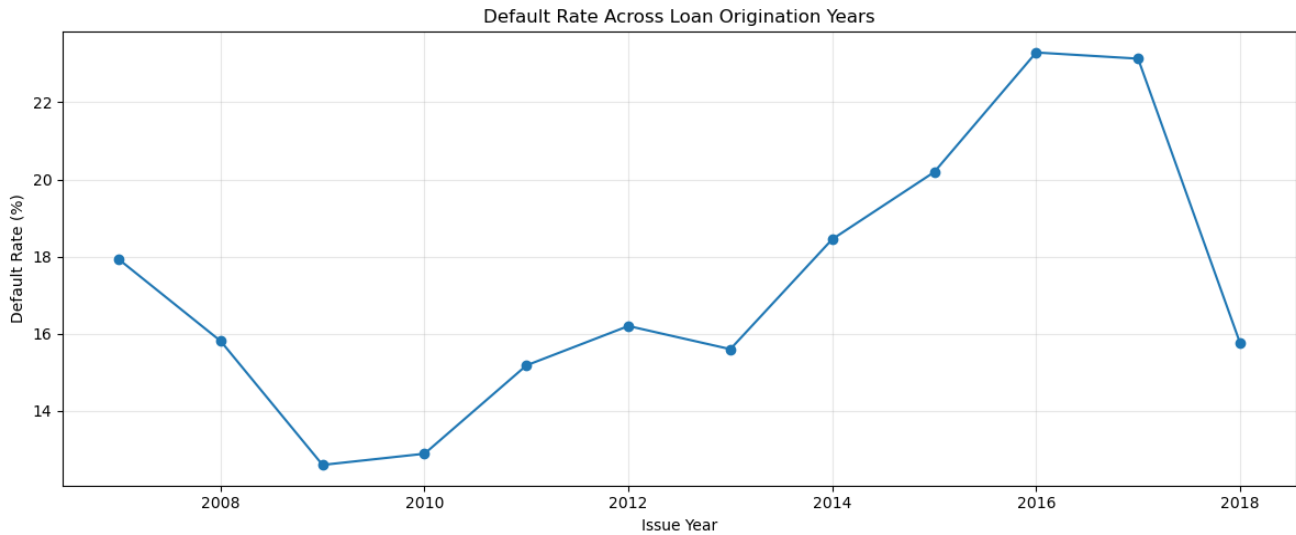


Figure 3.4. Default-rate trend by issue year

Figure 3.4 presents the observed default rate across issue years and makes year-to-year variation visible. The plot is used as a descriptive check of target behaviour before modelling. Because the project evaluates the model on future issue years, the figure helps show that the target distribution may vary across periods.

3.4 Feature Engineering

Table 3.2. GraphSAGE feature set

No.	GraphSAGE feature
1	loan_amnt
2	int_rate
3	installment
4	term_months
5	emp_length_years
6	annual_inc_log
7	dti
8	delinq_2yrs
9	fico_avg
10	inq_last_6mths
11	open_acc
12	pub_rec
13	revol_bal_log
14	revol_util
15	total_acc
16	tot_cur_bal_log
17	total_rev_hi_lim
18	acc_open_past_24mths

19	avg_cur_bal
20	bc_util
21	credit_history_years
22	mort_acc
23	loan_to_income
24	installment_to_income
25	revolving_utilization_decimal
26	bc_utilization_decimal

Table 3.2 lists the 26 variables supplied to GraphSAGE. The feature set combines loan characteristics such as loan amount, interest rate and term with borrower and credit-history measures such as income, DTI, FICO, delinquencies, account counts and utilization. Engineered ratios such as loan-to-income and installment-to-income are also included, giving the graph model a mixture of original and scale-normalized financial information.

Important transformations are kept simple and interpretable. Term is converted from text to months. Average FICO is calculated from the lower and upper FICO range. Income and selected balance variables use $\log(1+x)$ transforms to reduce skew. Loan-to-income and installment-to-income ratios create scale-normalized indicators. Credit-history years are calculated from the issue date and earliest credit-line date. Utilization percentages are converted to decimal form.

$$FICO_{avg} = (FICO_{low} + FICO_{high}) / 2 \quad \dots(10)$$

$$Loan_to_income = Loan\ Amount / Annual\ Income \quad \dots(11)$$

$$Installment_to_income = (12 \times Installment) / Annual\ Income \quad \dots(12)$$

$$Annual_Income_log = \log(1 + Annual\ Income) \quad \dots(13)$$

3.5 Leakage Control

Target and post-origination variables are excluded from the GraphSAGE input matrix. The audit checks that default, issue_date and issue_year are not among the 26 model features. The final notebook reports that the target variable and time fields were excluded and that no keyword-based leakage flags remained.

3.6 Missing Values and Standardization

The graph feature matrix initially contains missing values after transformation. The implementation calculates medians using training nodes, fills missing values and standardizes the resulting feature matrix. The final validation reports zero NaN values and zero infinite values. This step is essential because missing or infinite values can propagate through neural-network layers and destabilize training.

3.7 KNN Graph Construction

Table 3.3. Graph construction statistics

Statistic	Value
Nodes	30,000
Features per node	26
KNN neighbours	8
Initial directed edges	240,000
Final undirected edge entries	369,098
Minimum degree	8
Maximum degree	43
Average degree	12.3033
Isolated nodes	0
Self-loops	0

Table 3.3 quantifies the similarity graph used by GraphSAGE. The experiment contains 30,000 nodes, 26 features per node and $k=8$ nearest neighbours. This produces 240,000 initial directed edges, while the final representation contains 369,098 undirected edge entries, with degrees ranging from 8 to 43 and an average degree of 12.3033. The absence of isolated nodes and self-loops indicates that every sampled node participates in the constructed similarity structure.

With $k=8$, the initial directed edge count is $30,000 \times 8 = 240,000$. Reverse edges are added and duplicate pairs are removed, producing 369,098 edge entries in the PyTorch Geometric representation. The average degree is approximately 12.3033.

$$\text{Initial directed edges } (E_{\text{initial}}) = N \times k \quad \dots(14)$$

$$N \times k = 30,000 \times 8 = 240,000$$

3.8 GraphSAGE Model

Table 3.4. GraphSAGE configuration

Parameter	Value
Input features	26
Hidden channels	64
Layers	3
Output classes	2
Dropout	0.30
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Maximum epochs	100
Early stopping patience	15

Device	CPU
Trainable parameters	11,906

Table 3.4 summarizes the final GraphSAGE architecture and training settings. The model receives 26 input features, uses three layers with 64 hidden channels, 0.30 dropout and two output classes. Adam is used with learning rate 0.001 and weight decay 0.0001, with a maximum of 100 epochs and patience of 15 for early stopping. The trained network contains 11,906 parameters, and training was stopped at epoch 29 after the best validation loss of 0.8122 was retained.

Weighted cross-entropy is used because the training sample contains 16,315 non-default nodes and 3,685 default nodes. The resulting class weights are approximately 0.6129 for non-default and 2.7137 for default.

$$w_c = N / (C \times N_c) \quad \dots(15)$$

Training stopped at epoch 29 with a best validation loss of 0.8122. The early stopping mechanism retained the best model state rather than simply using the final epoch.

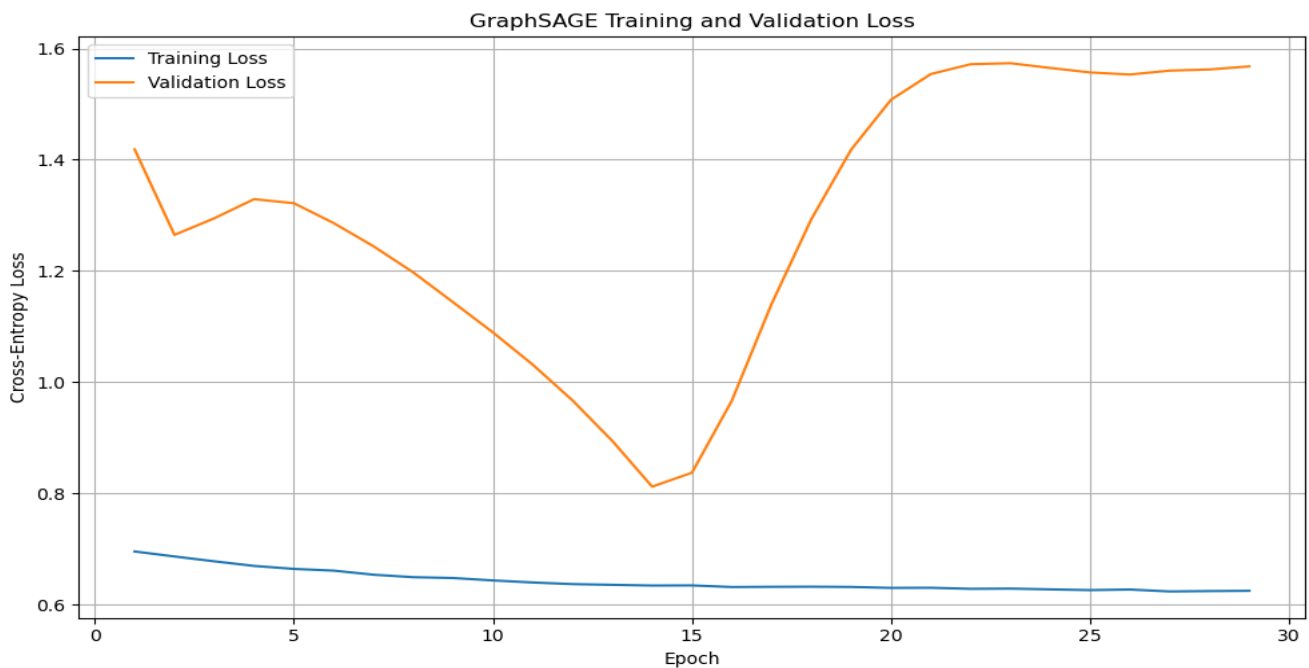


Figure 3.5. GraphSAGE training and validation loss

Figure 3.5 visualizes the training process by plotting loss on the training and validation sets across epochs. The curves are used to monitor whether the model continues to improve on unseen validation nodes or begins to overfit the training sample. The reported procedure selected the best validation state at epoch 29, with validation loss 0.8122, rather than simply taking the final epoch.

3.9 MLP Baseline

Table 3.5. MLP baseline configuration

Layer	Configuration
Input	26 features
Hidden 1	64 neurons + ReLU + dropout 0.30
Hidden 2	64 neurons + ReLU + dropout 0.30
Output	2 classes

Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Maximum epochs	100
Early stopping patience	15
Trainable parameters	6,018

Table 3.5 gives the configuration of the non-graph baseline used for comparison. The MLP uses the same 26 input features, two hidden layers of 64 neurons with ReLU and 0.30 dropout, and a two-class output. Its 6,018 trainable parameters and identical optimizer settings make the comparison useful because the main methodological difference is the absence of graph-edge information.

The MLP training stopped at epoch 16 with best validation loss 1.8240. The model uses the same class counts and class-weight formula as GraphSAGE but does not use edge information.

3.10 Threshold Selection

Thresholds from 0.10 to 0.90 are tested on validation data. For GraphSAGE, the selected threshold is 0.50 with validation F1-score 0.4657. Applying this threshold to the test set gives the final confusion matrix used in the report.

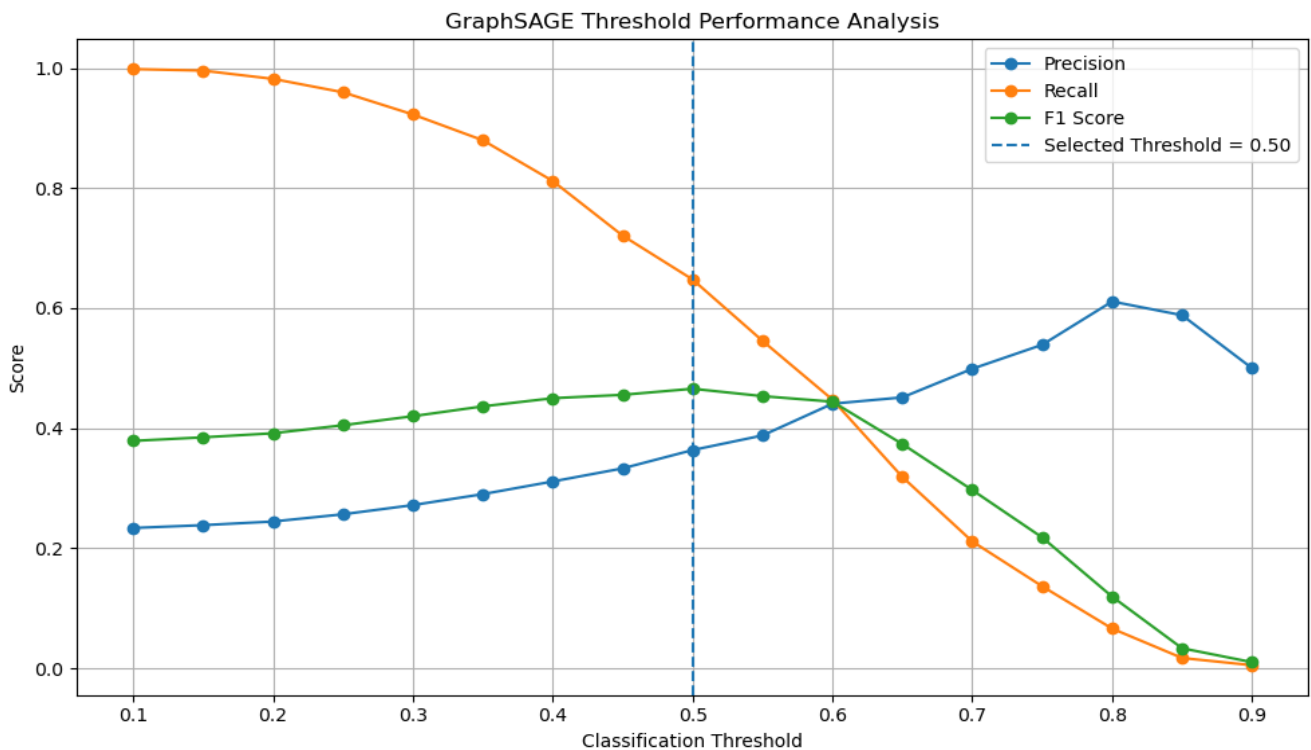


Figure 3.6. GraphSAGE threshold performance analysis

Figure 3.6 shows the validation performance obtained when the GraphSAGE classification threshold is varied from 0.10 to 0.90. The selected threshold is 0.50, where the validation F1-score is 0.4657. This threshold is then kept fixed for the test evaluation, linking threshold selection to the final confusion matrix and classification metrics in Chapter 5.

3.11 Probability Calibration

Validation probabilities are transformed into log-odds, and a logistic calibrator is fitted on the validation data. The fitted calibrator is then applied to the test probabilities. Reliability diagrams are generated with ten uniform bins.

$$\text{logit}(p) = \ln[p/(1-p)] \quad \dots(16)$$

The Brier score before and after calibration is the main numerical calibration measure reported in this project.

3.12 System Workflow

The complete FinTwinAI methodology follows an end-to-end workflow that connects data preparation, financial feature engineering, graph construction, machine learning, probability calibration, explainability, digital-twin generation, macroeconomic stress testing and dashboard visualization.

The first stage begins with the LendingClub-derived loan dataset containing 2,260,701 records. The dataset is cleaned by removing irrelevant observations and focusing the binary modelling task on Fully Paid and Charged Off/Default outcomes.

The cleaned data is divided temporally rather than randomly. Loans issued through 2015 are used for training, 2016 is used for validation, and 2017 onward is used for testing. This preserves the chronological structure of the financial data.

Next, financially meaningful variables are transformed into a 26-feature model matrix. Important transformations include average FICO calculation, logarithmic transformation of skewed financial variables, loan-to-income ratio, installment-to-income ratio, credit-history duration and utilization measures.

The model matrix is then standardized after missing values are filled using training-data medians. A K-nearest-neighbour graph with $k = 8$ is constructed using a 30,000-node sample. The initial directed graph contains:

$$E_{\text{initial}} = N \times k \quad \dots(14)$$

$$E_{\text{initial}} = 30,000 \times 8 = 240,000$$

After adding reverse edges and removing duplicate pairs, the final graph representation contains 369,098 edge entries, with an average degree of approximately 12.3033.

The graph is passed to a three-layer GraphSAGE model. A conventional MLP model is trained using the same feature matrix as a baseline without graph relationships. Both models are evaluated using accuracy, precision, recall, F1-score, ROC-AUC and average precision.

The validation set is also used for threshold selection and probability calibration. For GraphSAGE, the selected classification threshold is 0.50, with a validation F1-score of 0.4657. Logistic calibration is subsequently applied to improve the reliability of predicted probabilities.

The calibrated probability of default becomes an important input to the FinTwinAI digital-twin layer. Each profile is associated with its estimated default probability, exposure, expected loss, health/risk score, risk category and macroeconomic stress information.

Expected loss is calculated using:

$$EL_i = Exposure_i \times PD_i \times LGD \quad \dots(17)$$

For the prototype, $LG D = 1$ is used, giving:

$$EL_i = Exposure_i \times PD_i \quad \dots(18)$$

The digital-twin profiles are then aggregated at portfolio level. Macroeconomic stress scenarios are applied using predefined probability multipliers. Finally, the outputs are presented through the Streamlit dashboard, allowing users to inspect model performance, financial health, portfolio risk, SHAP explanations, stress scenarios and illustrative survival analysis.

The workflow therefore provides a complete connection between the original financial dataset and the final interactive FinTwinAI dashboard, while maintaining a separation between trained machine-learning predictions, rule-based educational financial-health calculations, and illustrative scenario analysis.

CHAPTER 4: IMPLEMENTATION

4.1 Software Environment

Table 4.1. Software and libraries

Tool / Library	Role in FinTwinAI
Python	Core programming language
Jupyter Notebook	Experimentation and research workflow
pandas	Data loading, cleaning and tabular analysis
NumPy	Numerical operations
scikit-learn	Preprocessing, metrics, calibration and evaluation
PyTorch	Neural-network computation
PyTorch Geometric	Graph data and GraphSAGE
SHAP	Model explanation
Matplotlib	Static research graphs
Plotly	Interactive dashboard visualizations
Streamlit	Web dashboard
FRED / pandas-datareader	Macroeconomic data retrieval

4.2 Data-to-Dashboard Output Files

The notebook exports a structured collection of CSV and model files so that the dashboard does not need to repeat the complete research pipeline every time it starts. The main outputs include `final_model_comparison.csv`, `final_scenario_analysis.csv`, `shap_feature_importance_final.csv`, `fred_macro_data.csv`, `digital_twin_profiles_final.csv`, `monthly_portfolio_risk_summary.csv`, `graphsage_confusion_matrix.csv`, `graphsage_feature_importance.csv`, `graphsage_model.pt` and `dashboard_risk_profile.csv`.

4.3 Dashboard Architecture

The FinTwinAI dashboard is designed as an interactive interface that integrates the major components of the proposed financial digital twin into a single workflow. It connects user input, data processing, risk prediction, explainability, financial-health assessment, and recommendation modules. The architecture is designed to present complex model outputs in a simple and interpretable form for users. The major dashboard components and their functions are described in the following subsections.

4.3.1 Dashboard Landing Page and File Loading

The landing page introduces FinTwinAI as an explainable AI-driven personal financial digital twin and displays the project output files required by the dashboard. The screenshot demonstrates that the prepared CSV and model outputs are available to the website, showing the connection between the research pipeline and the deployed interface.

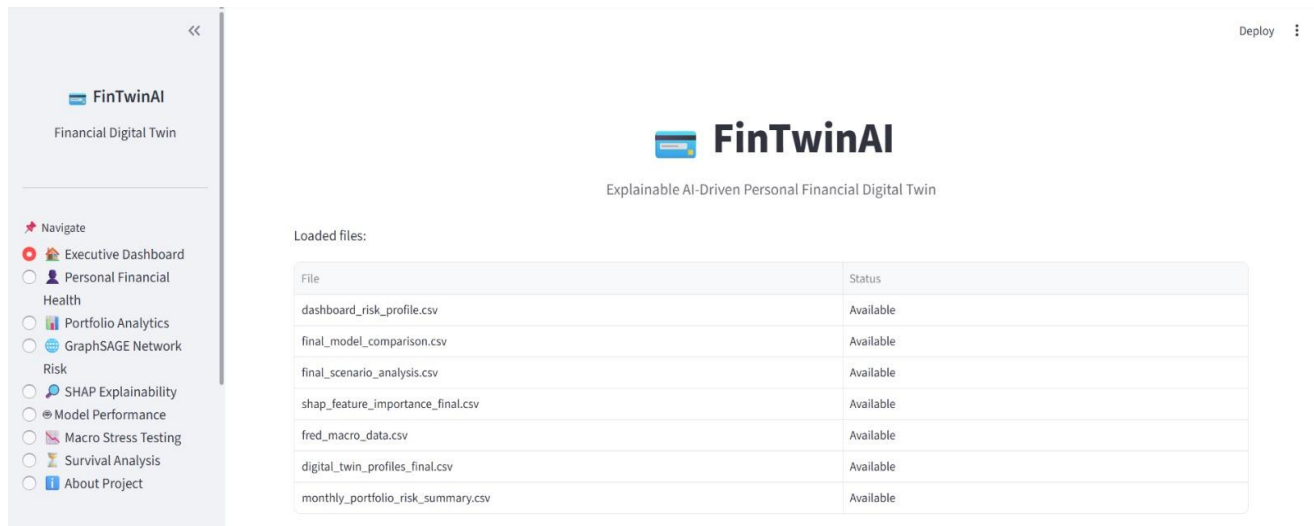


Figure 4.1. FinTwinAI dashboard landing page and loaded project files

4.3.2 Income and Loan Details Interface

The Personal Financial Health interface is designed for a normal user rather than a machine-learning specialist. The user enters monthly income, existing EMIs, loan amount, monthly living expenses, annual interest rate, credit score/FICO, credit utilization, delinquencies, employment experience and loan term. The values shown in the screenshot represent an example scenario used to demonstrate the working of the website.

Figure 4.2. Income and Loan Details input interface

The Streamlit source code defines a wide layout, an expanded sidebar, reusable data-loading functions, INR and percentage formatting functions, a risk-category function, a health-score function, an EMI calculator and multiple interactive pages. The dashboard explicitly labels itself as an academic prototype and warns users not to treat it as a formal credit approval or financial-advice system.

Table 4.2. Dashboard modules

Dashboard page	Main function
Executive Dashboard	Project KPIs, model comparison, calibration and pipeline
Personal Financial Health	EMI, debt ratios, remaining cash and educational health score

Portfolio Analytics	Risk-category counts, exposure and expected loss
GraphSAGE Network Risk	Graph statistics and confusion matrix
SHAP Explainability	Feature-attribution and importance view
Model Performance	MLP vs GraphSAGE metrics and calibration
Macro Stress Testing	Stress scenarios and macro indicators
Survival Analysis	Illustrative survival curve and scenario controls
About Project	Project identity, methods and limitations

4.4 Personal Financial Health Module

The website also contains a rule-based personal financial-health calculator. It accepts monthly income, loan amount, interest rate, loan term, existing EMI, monthly living expenses, credit score, revolving utilization, delinquencies and employment experience. This module is intentionally separate from the trained GraphSAGE model.

$$EMI = P.r[(1+r)^n / (1+r)^{n-1}] \quad \dots(19)$$

For a zero-interest loan, $EMI=P/n$. Total repayment is $EMI \times n$ and total interest is total repayment– P . The dashboard then calculates debt-to-income, EMI-to-income, loan-to-annual-income and remaining monthly cash.

$$Debt_to_income = 100 \times (Existing\ EMI + New\ EMI) / Monthly\ Income \quad \dots(20)$$

$$EMI_to_income = 100 \times (New\ EMI / Monthly\ Income) \quad \dots(21)$$

$$Loan_to_annual_income = Loan\ Amount / (12 \times Monthly\ Income) \quad \dots(22)$$

$$Remaining\ monthly\ cash = Monthly\ Income - Living\ Expenses - Existing\ EMI - New\ EMI \quad \dots(23)$$

The educational health score is calculated from a capped rule-based risk estimate. The dashboard uses Health Score = $(1 - Educational\ Risk) \times 100$. This score must not be confused with the GraphSAGE probability of default.

4.4.1 Loan Repayment Summary

After the user generates the report, the dashboard calculates the EMI, total repayment and total interest using the standard loan-amortization formula. The screenshot shows a ₹300,000 example loan at 11.50% annual interest for 36 months, producing an EMI of approximately ₹9,892.80 and total repayment of approximately ₹356,140.87. These figures demonstrate the numerical portion of the website and connect the mathematical formula to the user interface.

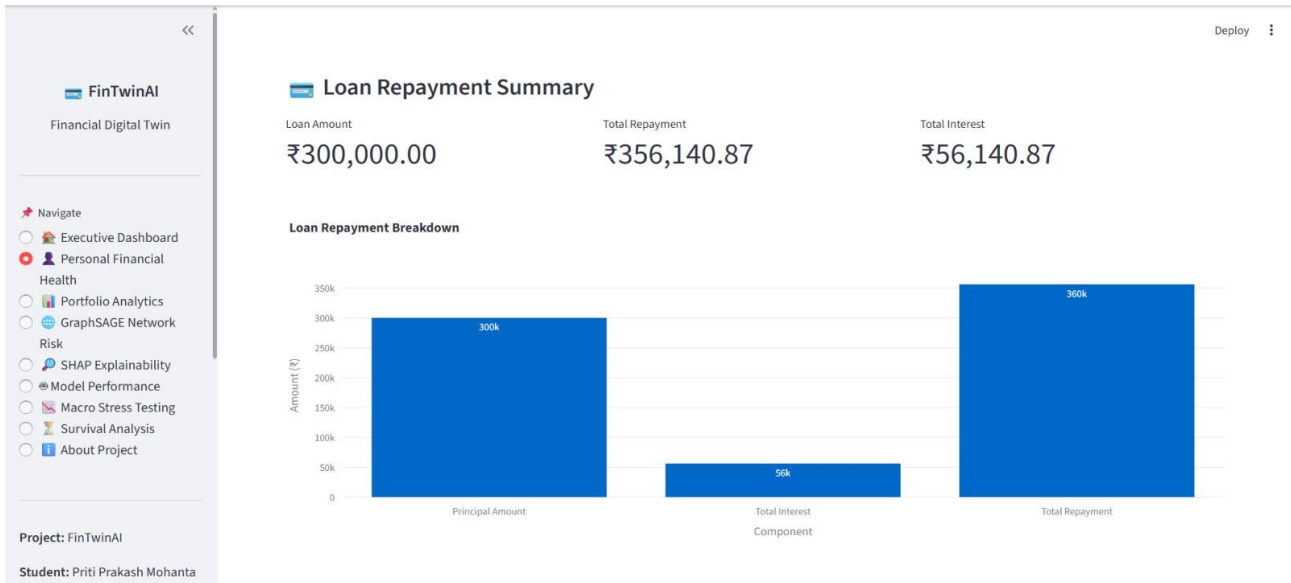


Figure 4.3. Loan Repayment Summary generated by FinTwinAI

4.4.2 Explanation in Simple Language

The website translates numerical results into plain language. In the demonstrated scenario, the system reports the estimated monthly EMI, loan duration, debt-to-income ratio and remaining monthly cash after expenses and debt payments. The interface also states that the displayed health score is an educational, rule-based indicator and is not the trained GraphSAGE prediction or a certified credit rating. This makes the output understandable while preserving a clear responsible-use boundary.

Explanation in Simple Language

Your estimated monthly EMI is ₹9,892.80.

This means you may need to pay approximately this amount every month for 36 months.

Your calculated debt-to-income ratio is 35.6%. This compares your monthly debt payments with your monthly income.

Your estimated remaining cash after expenses and debt payments is ₹5,107.20 per month.

The displayed health score is an educational, rule-based indicator. It is **not the trained GraphSAGE model's prediction** and should not be treated as a certified credit rating.

Disclaimer: This assessment is for academic and educational purposes only. Actual loan eligibility, interest rates and credit decisions depend on financial institutions and verified borrower information.

Figure 4.4. Explanation in Simple Language and educational health interpretation

4.4.3 Financial Indicators and Recommendations

The final part of the personal-health report summarizes EMI-to-income ratio, debt-to-income ratio, loan-to-annual-income ratio, credit utilization, credit score and recent delinquencies. The dashboard then provides simple educational recommendations such as preparing a repayment budget and building an emergency fund before increasing debt. The interface also contains a disclaimer that actual eligibility, interest rates and credit decisions depend on financial institutions and verified borrower information.

Important Financial Indicators

Indicator	Value
EMI-to-Income Ratio	14.13%
Debt-to-Income Ratio	35.56%
Loan-to-Annual-Income Ratio	0.36
Credit Utilization	21.00%
Credit Score	700
Delinquencies	0

Personalized Recommendations

-  Your debt burden is moderate to high. Prepare a monthly repayment budget.
-  Build an emergency fund before increasing your debt.

Figure 4.5. Important Financial Indicators and personalized recommendations

4.5 Digital-Twin Profile

Each final profile contains Issue_Date, Default_Probability, Exposure, Expected_Loss, Month, MSI, Stress_Level, Health/Risk Score, Exposure_Band and Risk_Category. The final dashboard dataset contains 5,000 records and no missing values in the exported dashboard fields.

4.6 Risk Categories

Table 4.3. Risk-category thresholds

Probability of Default	Category
$p < 0.20$	Low Risk
$0.20 \leq p < 0.40$	Medium Risk
$0.40 \leq p < 0.60$	High Risk
$p \geq 0.60$	Very High Risk

For clarity, the final report uses “Financial Health Score” for the inverse probability scale used by the dashboard: Health Score = $(1-p) \times 100$. This resolves the naming ambiguity between “risk score” and “health score” present in different intermediate notebook cells.

4.7 Explainability Implementation

The notebook performs SHAP calculations and also computes GraphSAGE input-gradient importance. The exported GraphSAGE gradient importance ranks Feature_2, Feature_23, Feature_4, Feature_20 and Feature_25 among the first five in the displayed output. A separate SHAP table reports Feature_16, Feature_1, Feature_9, Feature_25 and Feature_15 among the top mean-absolute contributions. Because the intermediate labels are anonymized, the final deployment version should join these values with the recovered original feature names before presenting them as financial explanations.

4.8 Survival Module

The dashboard's survival module is implemented as an illustrative exponential curve. The hazard depends on health score and stress level. The implementation uses a base hazard of 0.015 and computes survival probability with an exponential function. This is a pedagogical visualization, not a validated survival model.

$$\lambda = 0.015 \times [1 + (100 - \text{Health Score})/100 + (\text{Stress Level}/100)] \quad \dots(24)$$

$$S(t) = e^{(-\lambda t)} \quad \dots(9)$$

$$\text{Event Probability } (t) = 1 - S(t) \quad \dots(25)$$

4.9 Responsible-Use Controls

- The dashboard states that the project is an academic prototype.
- Loan outputs are not presented as official credit approval decisions.
- The KNN graph is explicitly described as feature similarity rather than a real borrower network.
- Stress multipliers are labelled as hypothetical assumptions.
- The survival page is labelled illustrative because event-time data were not validated.
- Currency is not assumed when the original dataset unit is unknown.

CHAPTER 5: RESULTS AND ANALYSIS

5.1 Final Model Performance

Table 5.1. Final model performance comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Avg. Precision
MLP Baseline	0.7430	0.2430	0.0977	0.1393	0.5533	0.2417
GraphSAGE	0.6432	0.3221	0.6113	0.4219	0.6858	0.3560

Table 5.1 reports the main test-set metrics for the two models. The MLP records 74.30% accuracy but only 9.77% default recall, while GraphSAGE records 64.32% accuracy and 61.13% default recall. GraphSAGE also has F1-score 42.19%, ROC-AUC 0.6858 and average precision 0.3560, compared with 13.93%, 0.5533 and 0.2417 for the MLP. These values describe different error profiles rather than a single measure of model quality.

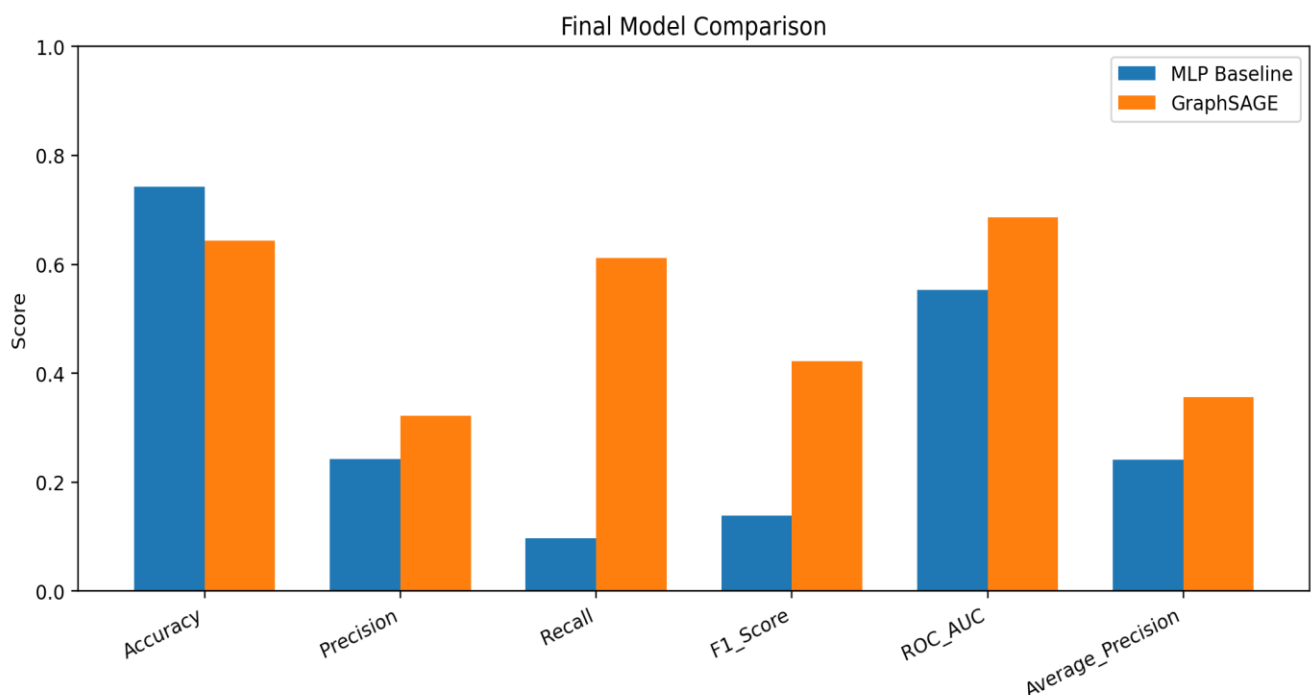


Figure 5.1. Final MLP versus GraphSAGE performance comparison

Figure 5.1 provides a visual comparison of the performance values in Table 5.1. The chart makes the contrast between overall accuracy and default-sensitive measures clear: the MLP reaches 74.30% accuracy, whereas GraphSAGE reaches 61.13% recall for defaults and 42.19% F1. The figure helps interpret why accuracy alone does not fully describe performance under the reported class imbalance.

The two models show different error profiles. The MLP has higher overall accuracy because it predicts the majority class more effectively, but its default recall is only 9.77%. GraphSAGE has lower accuracy at 64.32%, but its default recall is 61.13%, F1-score is 42.19%, ROC-AUC is 0.6858 and average precision is 0.3560. The result shows why a single accuracy number is not enough for an imbalanced risk problem.

5.2 Additional Classification Metrics

Table 5.2. Additional classification metrics

Model	Balanced Accuracy	Specificity	MCC
MLP	0.5077	0.9177	0.0224
GraphSAGE	0.6316	0.6518	0.2195

Table 5.2 adds balanced accuracy, specificity and Matthews correlation coefficient to the main evaluation. GraphSAGE has balanced accuracy 0.6316 and MCC 0.2195, while the MLP has 0.5077 and 0.0224 respectively. The MLP has higher specificity at 0.9177 compared with 0.6518 for GraphSAGE. These figures show the trade-off between correctly identifying the majority non-default class and capturing default cases.

Balanced accuracy and MCC provide additional information about the behaviour of the classifier. GraphSAGE's balanced accuracy of 0.6316 is higher than the MLP's 0.5077, while MLP specificity is higher. The Matthews correlation coefficient is also higher for GraphSAGE, although the absolute value remains moderate.

5.3 GraphSAGE Confusion Matrix

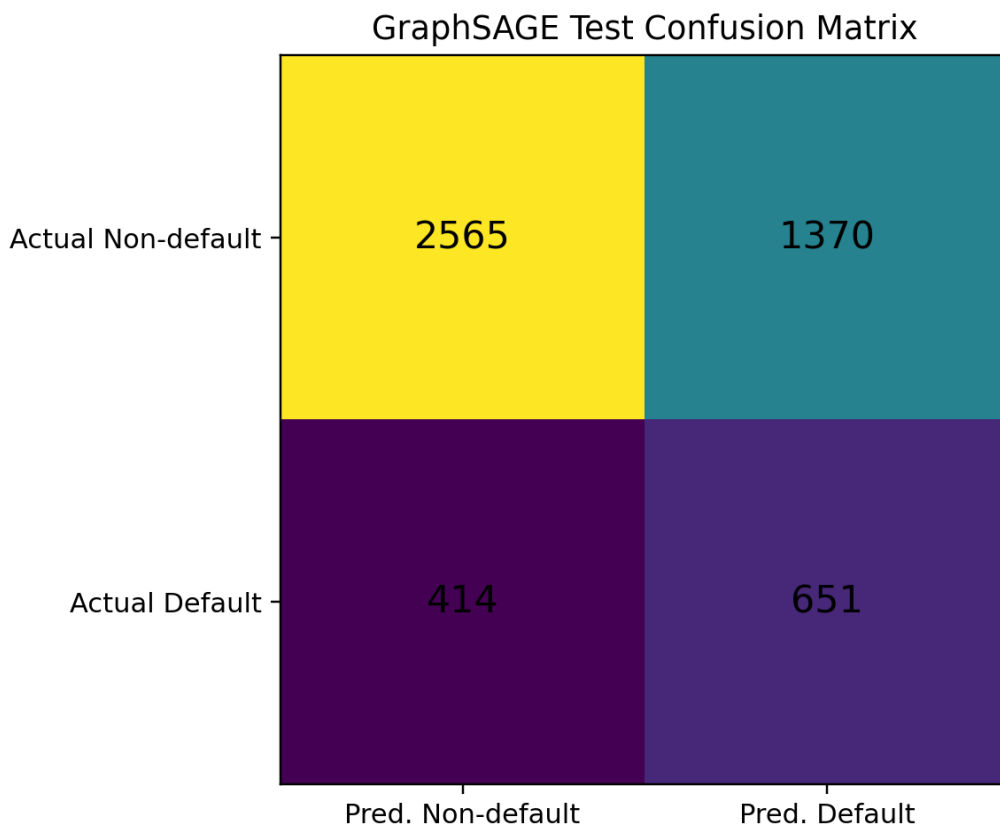


Figure 5.2. GraphSAGE test confusion matrix

Figure 5.2 breaks the 5,000 test cases into four outcomes: 2,565 true negatives, 1,370 false positives, 414 false negatives and 651 true positives. From these counts, default recall is $651/(651+414)=61.13\%$ and default precision is $651/(651+1,370)=32.21\%$. The 1,370 false positives show why the model is more suitable for screening or prioritization than automatic rejection.

The test set contains 5,000 cases: 2,565 true negatives, 1,370 false positives, 414 false negatives and 651 true positives. The default recall is $651/(651+414)=0.6113$. Precision for predicted default is

$651/(651+1,370)=0.3221$. The relatively large false-positive count means the model is more appropriate for screening or prioritization than automatic rejection.

5.4 ROC and Precision-Recall Analysis

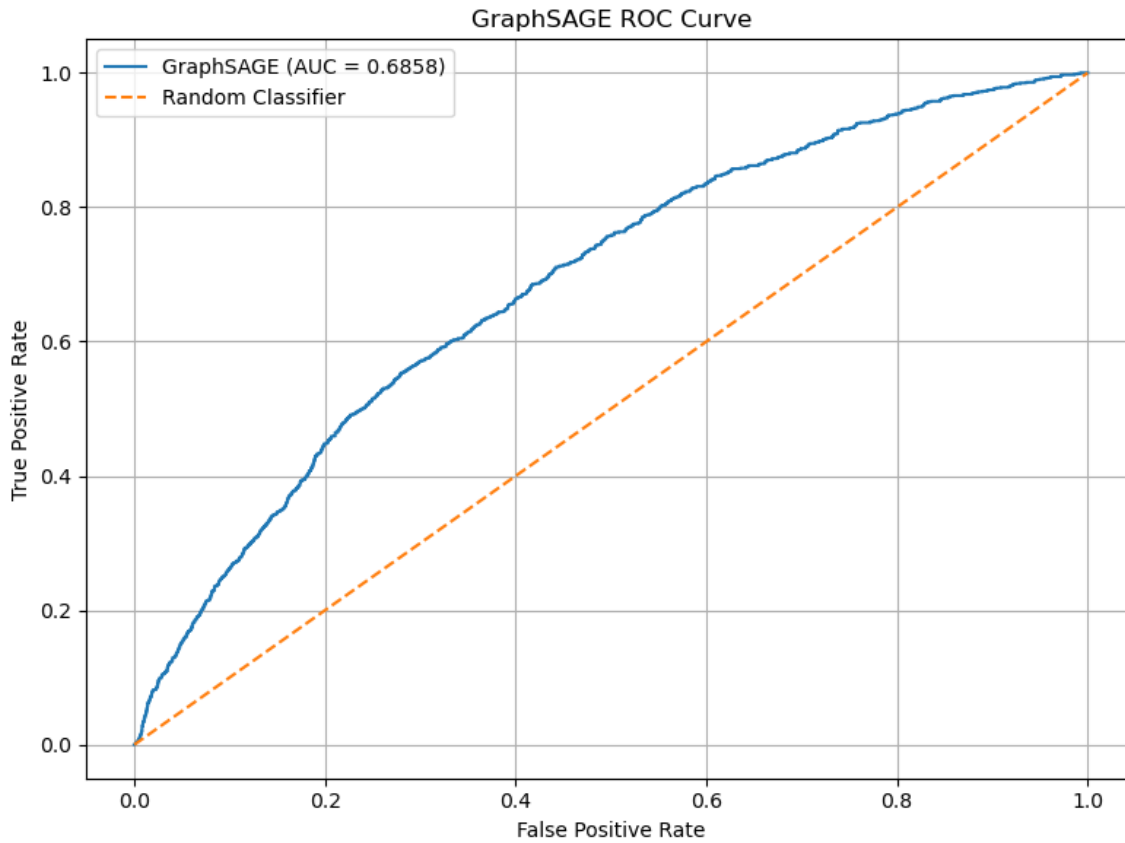


Figure 5.3. GraphSAGE ROC curve

Figure 5.3 plots the GraphSAGE true-positive rate against the false-positive rate over classification thresholds. Its reported ROC-AUC is 0.6858, providing a threshold-independent summary of class-separation behaviour on the test set. The curve is interpreted together with the chosen threshold rather than as a replacement for the final classification metrics.

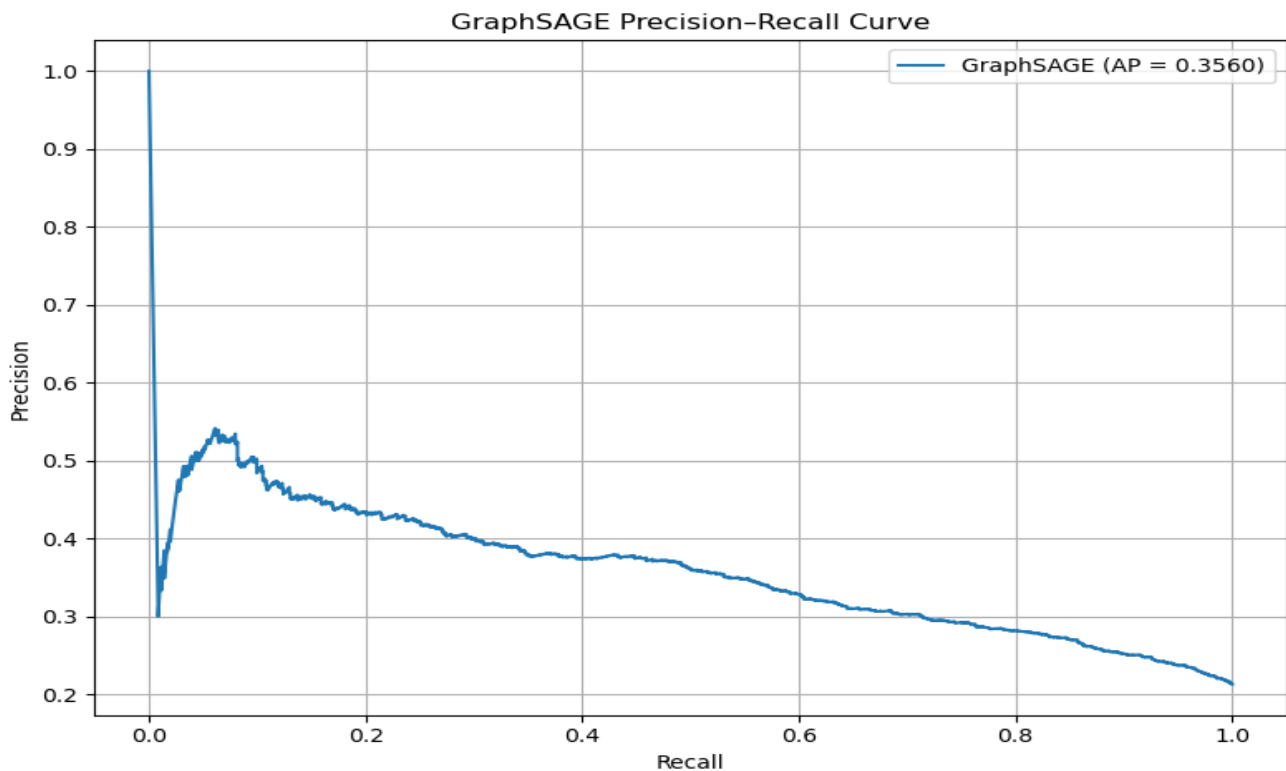


Figure 5.4. GraphSAGE precision-recall curve

Figure 5.4 focuses on precision and recall as the decision threshold changes. This view is particularly relevant because default is the less frequent class. The reported average precision for GraphSAGE is 0.3560, so the figure complements the ROC analysis by showing the relationship between identifying more defaults and maintaining precision.

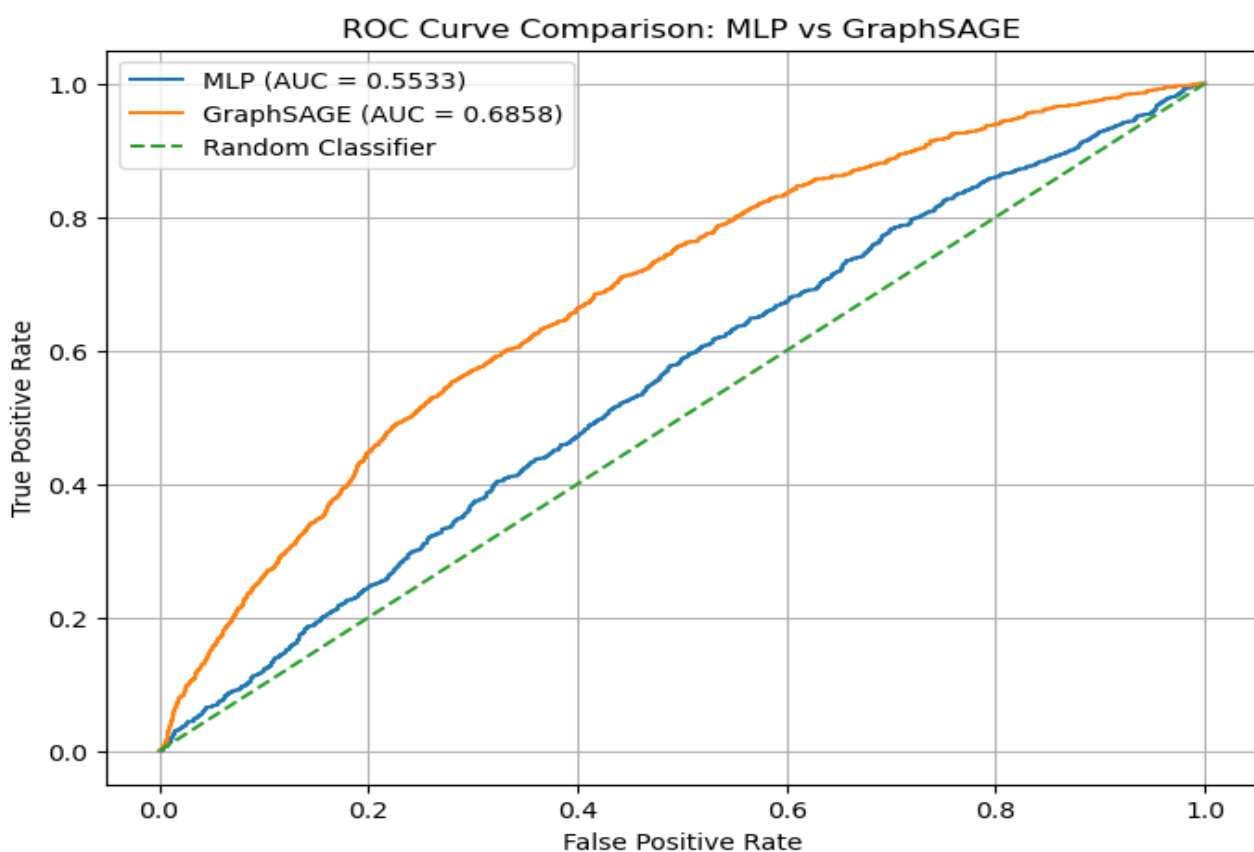


Figure 5.5. ROC comparison between MLP and GraphSAGE

Figure 5.5 places the ROC behaviour of both models on the same plot. The reported ROC-AUC values are 0.5533 for the MLP and 0.6858 for GraphSAGE. The visual comparison therefore shows the difference in threshold-independent separation measured on the same test setting.

ROC-AUC of 0.6858 indicates that GraphSAGE separates the two classes better than the MLP's 0.5533 on the reported test set. The precision-recall curve is especially relevant because the default class is less frequent. Average precision rises from 0.2417 for MLP to 0.3560 for GraphSAGE.

5.5 Probability Calibration

Table 5.3. Probability calibration results

Model	Before	After	Absolute reduction	Relative reduction
MLP	0.228459	0.167802	0.060657	26.55%
GraphSAGE	0.222188	0.158626	0.063562	28.61%

Table 5.3 quantifies the change in Brier score before and after probability calibration. For the MLP, the Brier score decreases from 0.228459 to 0.167802, an absolute reduction of 0.060657 or 26.55%. For GraphSAGE, it decreases from 0.222188 to 0.158626, an absolute reduction of 0.063562 or 28.61%. Because calibrated probabilities are later used for risk categories and expected loss, this table connects calibration directly to the digital-twin outputs.

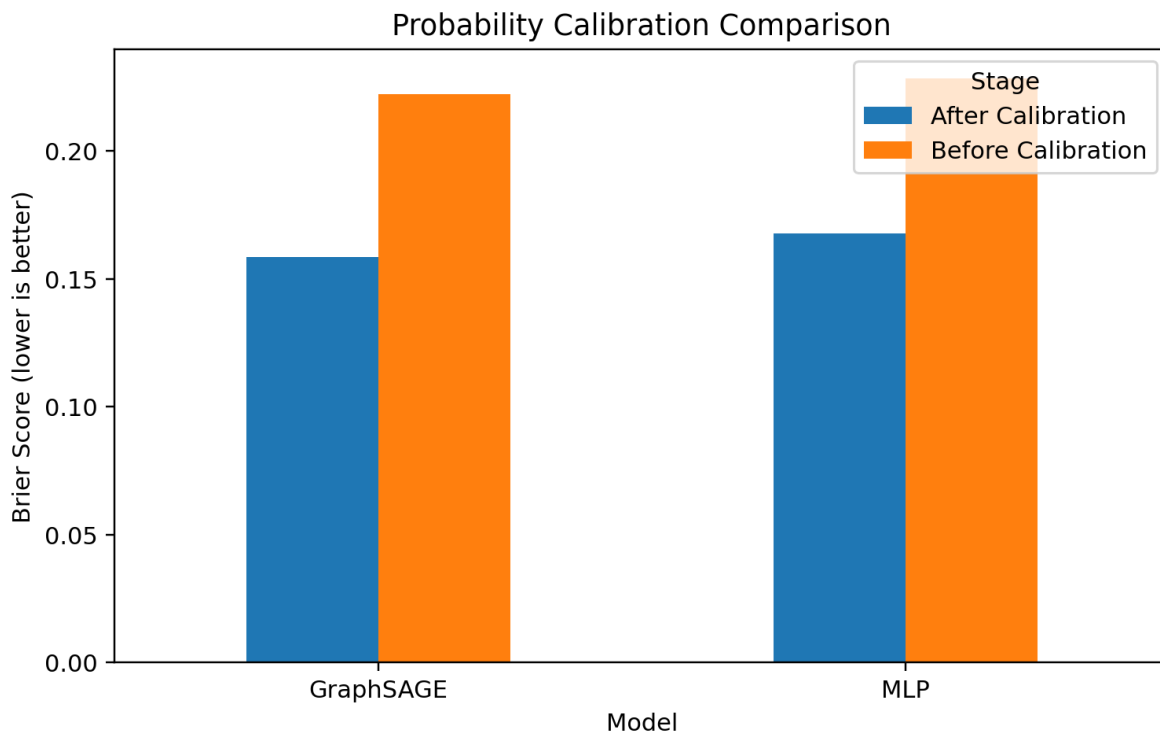


Figure 5.6. Probability calibration comparison

Figure 5.6 visualizes the pre- and post-calibration probability behaviour for both models. The main numerical change is the reduction in Brier score: GraphSAGE moves from 0.222188 to 0.158626, while MLP moves from 0.228459 to 0.167802. The plot illustrates the improvement in probability-error measure reported in Table 5.3 rather than an improvement in classification accuracy.

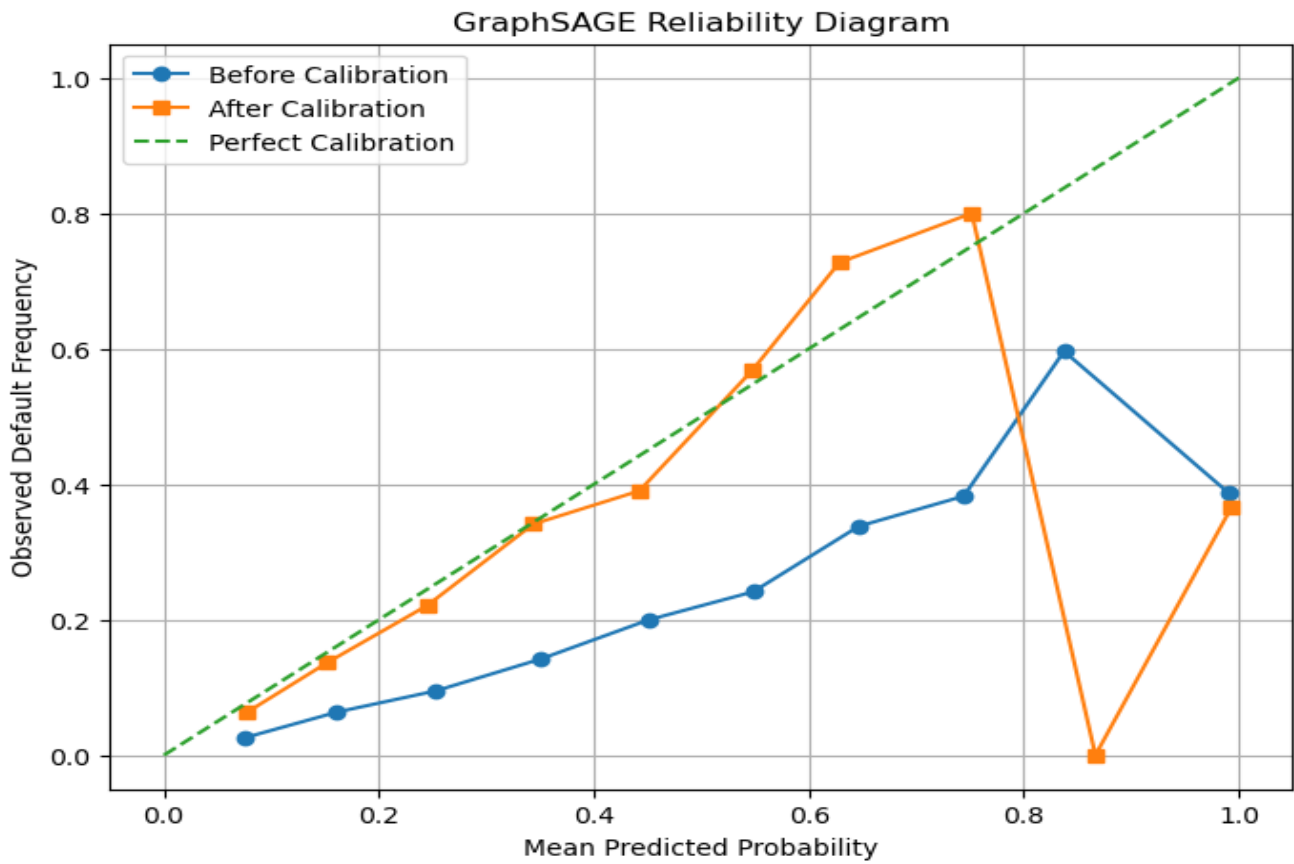


Figure 5.7. GraphSAGE reliability diagram

Figure 5.7 compares predicted probability bins with observed outcome frequencies for GraphSAGE. A reliability diagram is useful because the downstream digital-twin layer interprets the model output as a probability rather than only as a class label. The figure should be read alongside the post-calibration Brier score of 0.158626 and not as evidence of perfect operational calibration.

Calibration reduces the Brier score for both models. The GraphSAGE reduction is 0.063562, which is approximately 28.61% of the pre-calibration Brier score. This matters because FinTwinAI uses probability estimates downstream for risk categories and expected loss.

5.6 Calibrated Probability Distribution

Table 5.4. Calibrated probability summary

Model	Mean before	Mean after	Minimum after	Maximum after
MLP	0.4610	0.2333	0.2137	0.4246
GraphSAGE	0.4593	0.2354	0.0000	1.0000

Table 5.4 shows how calibration changes the average predicted probability and the observed range of calibrated outputs. The MLP mean changes from 0.4610 before calibration to 0.2333 after calibration, while GraphSAGE changes from 0.4593 to 0.2354. The GraphSAGE calibrated range extends from 0.0000 to 1.0000, demonstrating that calibration changes the probability scale substantially and that the values still require reliability assessment before operational interpretation.

The calibrated probabilities should still be interpreted with caution. A broad range from near zero to near one is not evidence of perfect calibration by itself. Reliability curves, out-of-time validation and subgroup analysis are needed before operational use.

5.7 Digital-Twin Portfolio Results

Table 5.5. Digital-twin portfolio summary

Risk Category	Loans	Average Health Score	Exposure	Expected Loss	Average MSI
Low Risk	400	15.2788	4,919,925	719,687.25	28.8300
Medium Risk	1,613	30.9351	20,099,325	6,234,576.74	28.8875
High Risk	1,791	49.5733	25,201,775	12,595,090.37	28.9266
Very High Risk	1,196	70.9448	22,637,825	16,311,737.66	28.8743
Total	5,000	—	72,858,850	35,861,092.02	28.89375

Table 5.5 summarizes the 5,000 digital-twin profiles by risk category. The portfolio has total exposure of 72,858,850 and baseline expected loss of 35,861,092.02. The High Risk group contains 1,791 loans with exposure 25,201,775 and expected loss 12,595,090.37, while the Very High Risk group contains 1,196 loans with exposure 22,637,825 and expected loss 16,311,737.66. The table demonstrates how individual probabilities are aggregated into portfolio-level financial quantities.

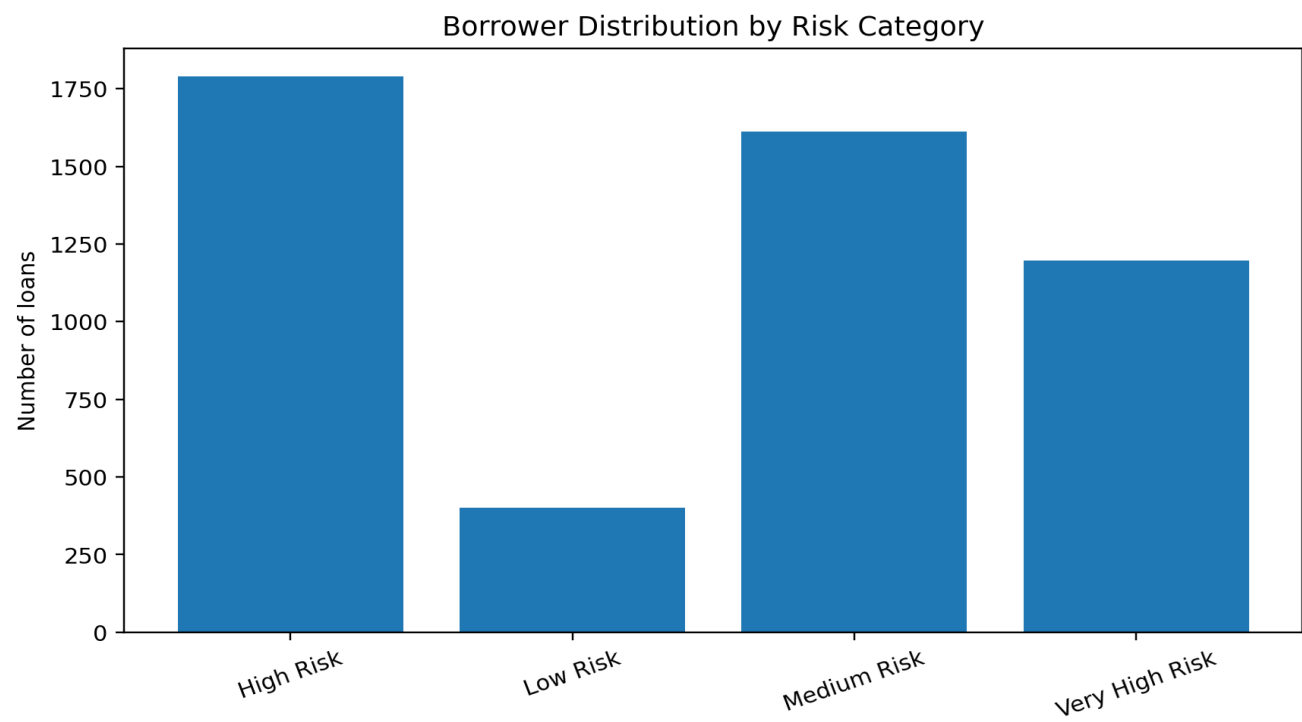


Figure 5.8. Borrower distribution by risk category

Figure 5.8 visualizes the number of profiles assigned to each risk category. There are 400 Low Risk, 1,613 Medium Risk, 1,791 High Risk and 1,196 Very High Risk profiles, summing to 5,000. The chart makes the concentration of profiles across the four probability bands easier to inspect than the table alone.

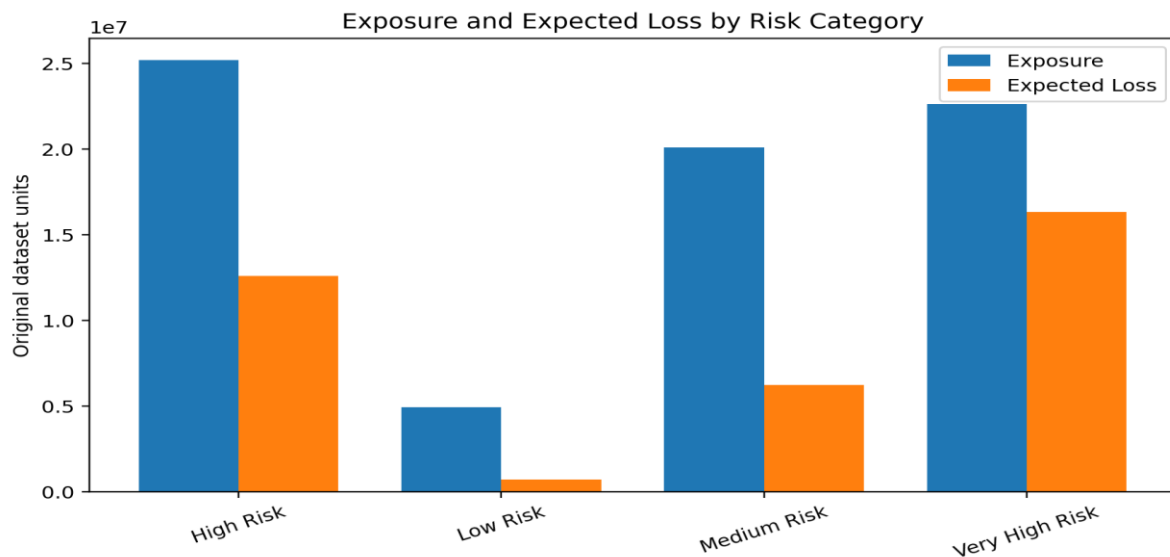


Figure 5.9. Exposure and expected loss by risk category

Figure 5.9 compares exposure and expected loss across the four risk groups. Very High Risk has expected loss of about 16.31 million, while High Risk has about 12.60 million and Medium Risk about 6.23 million. The exposure values explain why aggregate loss is not determined by borrower count or probability alone: High Risk has exposure of 25,201,775, greater than the 22,637,825 exposure in Very High Risk.

The final portfolio contains 5,000 digital-twin profiles. Medium Risk is the largest group with 1,613 loans, followed by High Risk with 1,791 loans and Very High Risk with 1,196 loans; Low Risk contains 400 loans. Expected loss is concentrated in the higher-risk groups. The Very High Risk group has expected loss of approximately 16.31 million, while High Risk has approximately 12.60 million.

The table also shows why portfolio risk cannot be judged from probability alone. High Risk has greater exposure than Very High Risk, so its aggregate expected loss is also large. This is the main value of the digital-twin layer: it translates model probability into a portfolio quantity that can be aggregated and monitored.

5.8 Macroeconomic Indicators and Stress Index

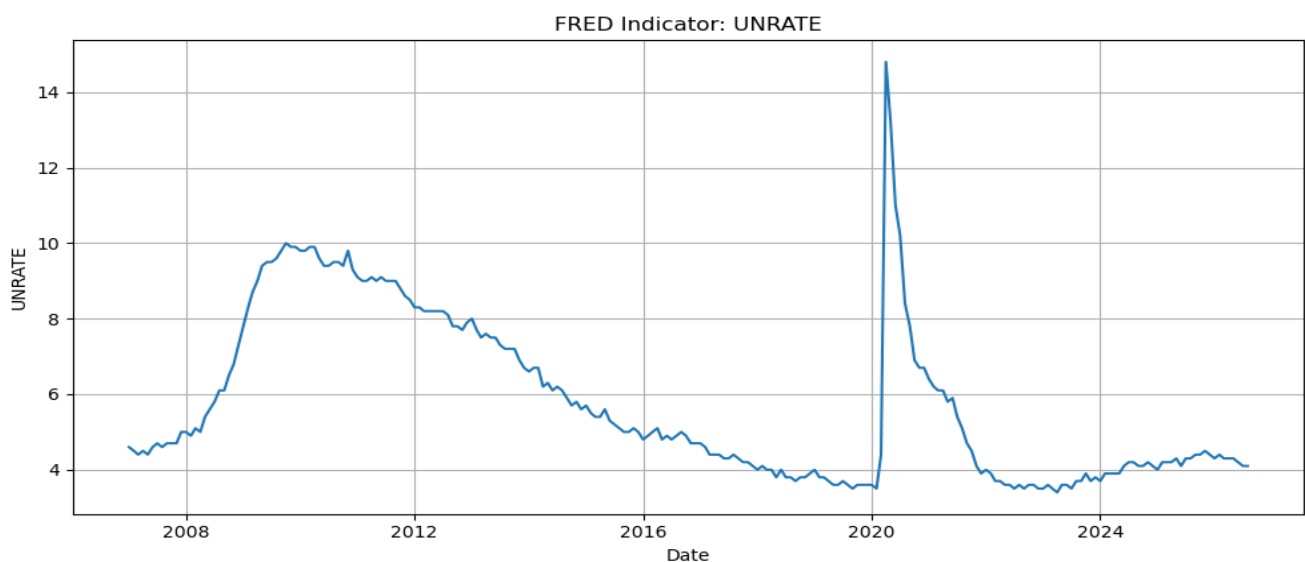


Figure 5.10A. FRED unemployment-rate series used in the macroeconomic module

Figure 5.10A shows the monthly unemployment-rate series used as one input to the macroeconomic module. It provides the employment-side component of the stress context and is later combined with other standardized macroeconomic variables to construct the illustrative MSI.

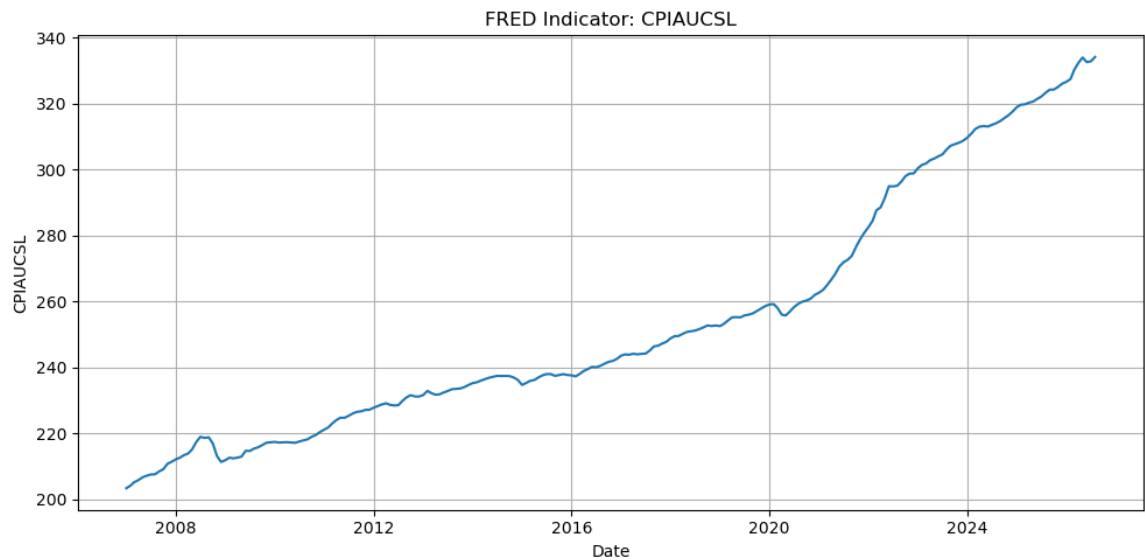


Figure 5.10B. FRED CPI series used in the macroeconomic module

Figure 5.10B presents the monthly CPI series used in the macroeconomic module. The series contributes information about price-level conditions; the notebook derives inflation-related information before standardization for the MSI calculation.

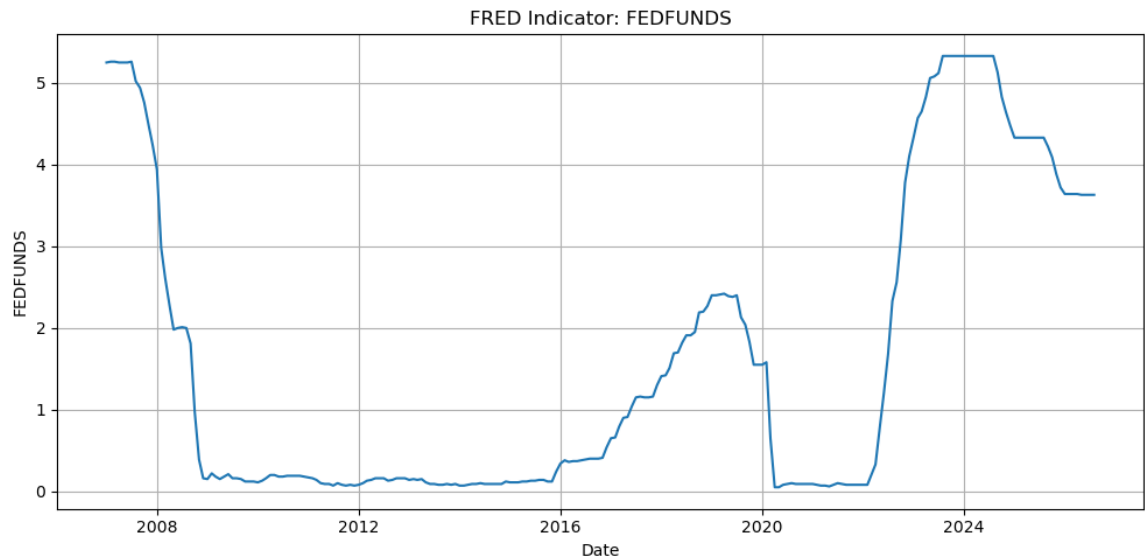


Figure 5.10C. FRED federal-funds-rate series used in the macroeconomic module

Figure 5.10C shows the federal-funds-rate series used as the interest-rate component of the macroeconomic input. Together with unemployment, CPI and GDP information, it provides a broader set of economic indicators for the illustrative stress index.

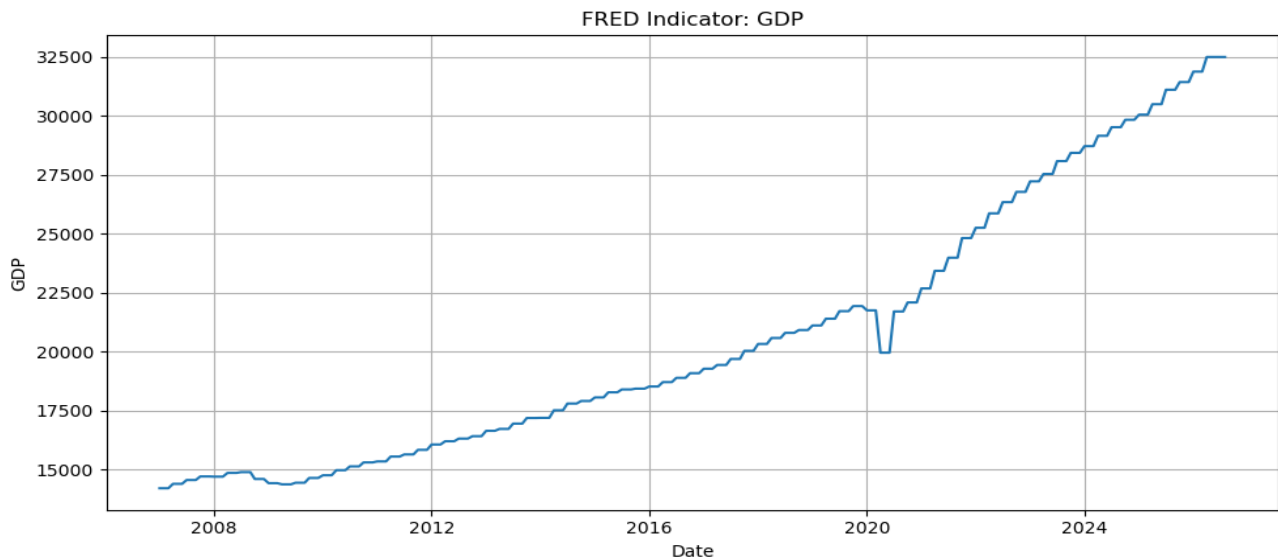


Figure 5.10D. FRED GDP series used in the macroeconomic module

Figure 5.10D presents the GDP series used in the macroeconomic module. The notebook derives GDP growth from the series and combines the standardized result with the other indicators. The complete macroeconomic dataset contains 236 monthly rows.

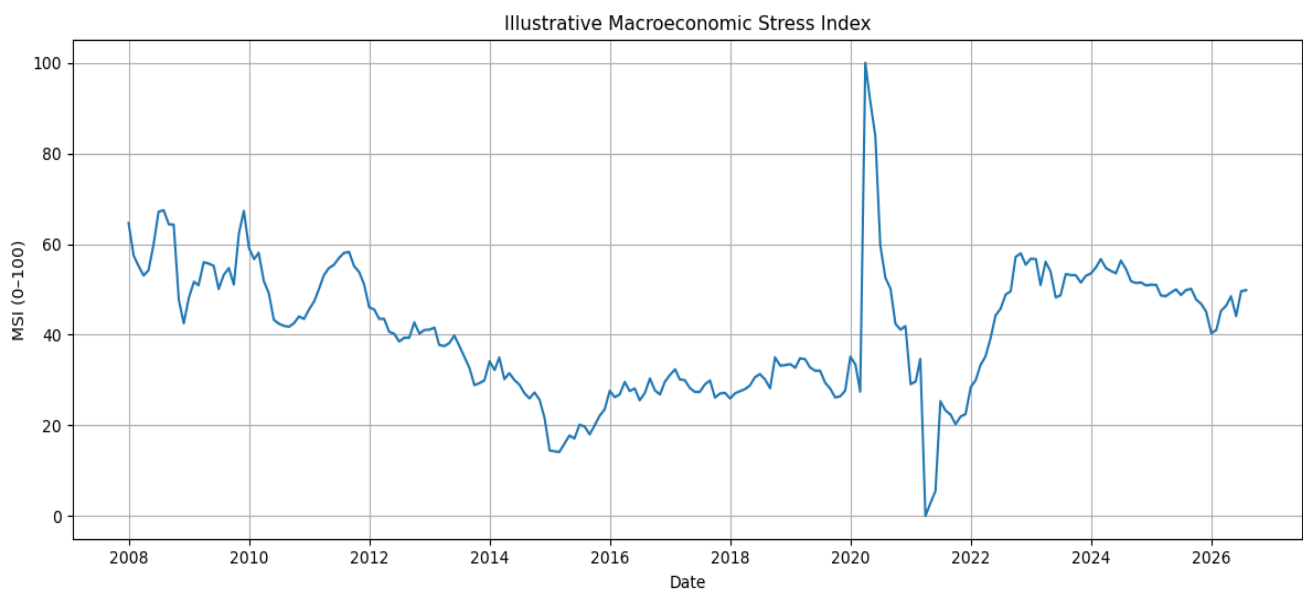


Figure 5.11. Illustrative Macroeconomic Stress Index

Figure 5.11 summarizes the illustrative Macro Stress Index across the available period. The underlying macroeconomic dataset contains 236 monthly observations of UNRATE, CPIAUCSL, FEDFUNDS and GDP, with derived inflation and GDP-growth measures. The final dashboard sample has an average MSI of approximately 28.89375; the index is a constructed analytical indicator rather than an observed market-risk measure.

The macroeconomic dataset contains 236 monthly rows and includes UNRATE, CPIAUCSL, FEDFUNDS and GDP. The notebook then derives inflation and GDP growth and standardizes selected variables to create an illustrative MSI. The final dashboard sample has an average MSI of approximately 28.89375.

5.9 Stress Scenario Analysis

Table 5.6. Stress scenario analysis

Scenario	PD multiplier	Expected loss	Increase	% increase	Loss / exposure
Baseline	1.00	35,861,092.02	0	0.0000%	49.219953%
Moderate Stress	1.20	42,763,119.00	6,902,026.98	19.246561%	58.693102%
Severe Stress	1.50	51,566,807.22	15,705,715.20	43.795976%	70.776312%

Table 5.6 quantifies the sensitivity of portfolio expected loss to the assumed probability multipliers. Baseline uses 1.00 and expected loss is 35,861,092.02. Moderate stress uses 1.20 and raises expected loss to 42,763,119.00, an increase of 6,902,026.98 or 19.25%. Severe stress uses 1.50 and raises expected loss to 51,566,807.22, an increase of 15,705,715.20 or 43.80%. These are scenario calculations based on assumptions, not observed future losses.

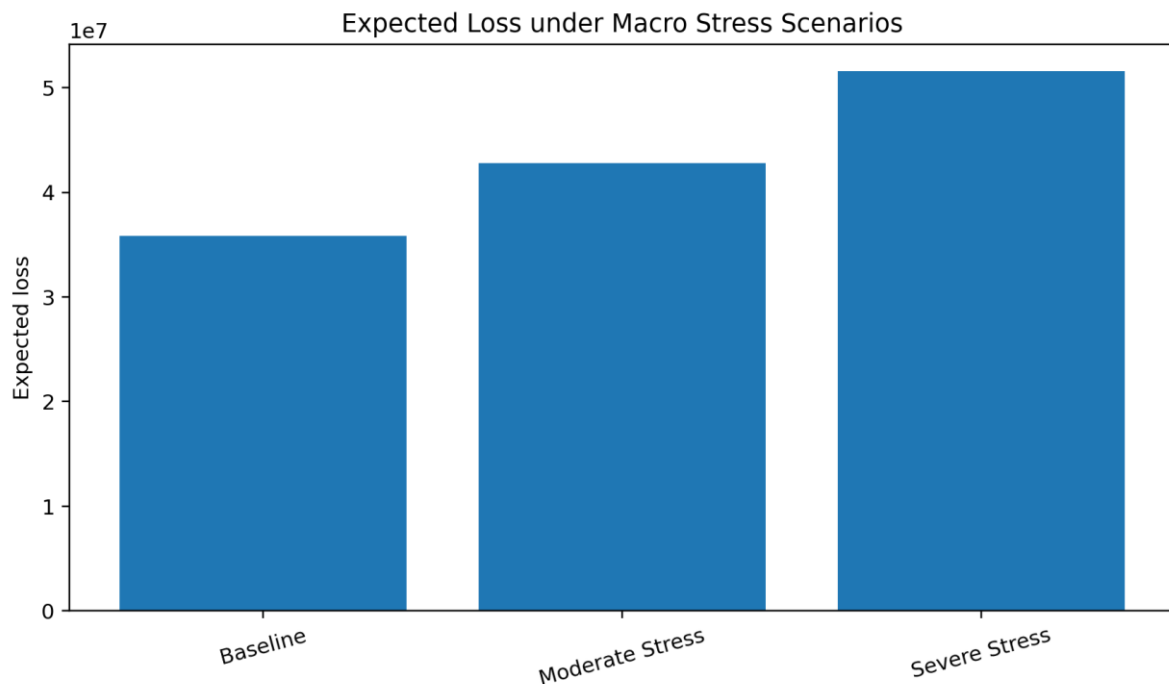


Figure 5.12. Expected loss under baseline, moderate and severe stress

Figure 5.12 visualizes the increase in portfolio expected loss across the three assumed scenarios. The baseline value is approximately 35.86 million, rising to 42.76 million under moderate stress and 51.57 million under severe stress. The widening bars reflect the direct effect of the 1.20 and 1.50 probability multipliers used in the sensitivity analysis.

The moderate scenario increases expected loss by approximately 6.90 million, or 19.25%, relative to baseline. Severe stress increases expected loss by approximately 15.71 million, or 43.80%. These results follow directly from the assumed PD multipliers and therefore represent sensitivity analysis rather than observed losses.

$$\text{Percentage increase} = [(EL_{\text{scenario}} - EL_{\text{baseline}}) / EL_{\text{baseline}}] \times 100 \quad \dots(26)$$

5.10 Monthly Portfolio Results

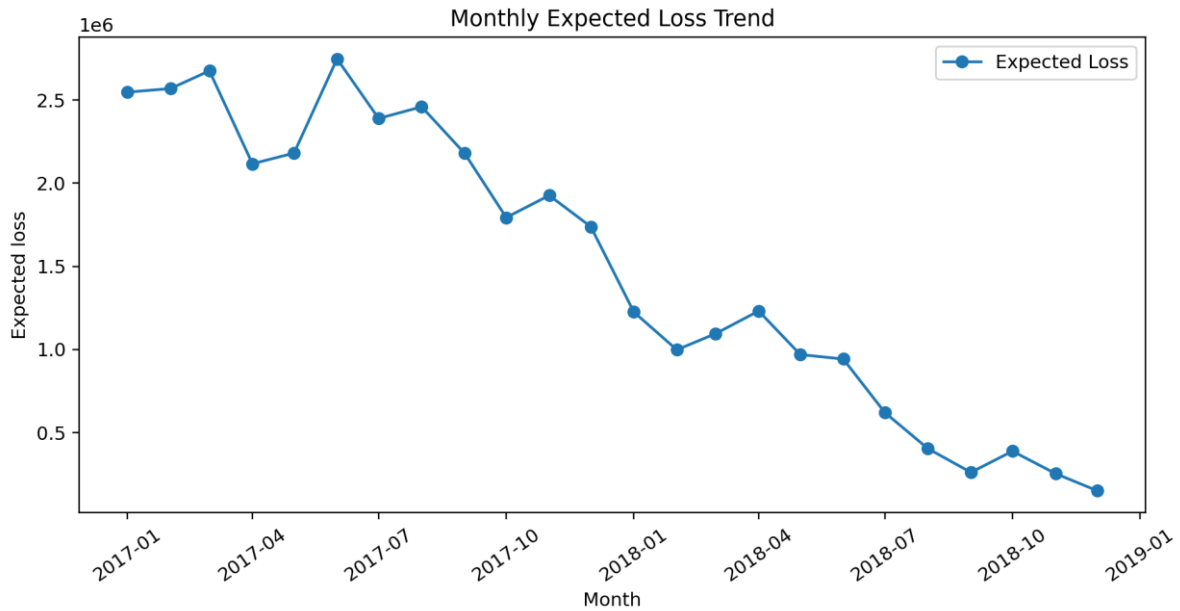


Figure 5.13. Monthly baseline expected-loss trend

Figure 5.13 shows the baseline expected-loss value for each of the 24 monthly portfolio observations from January 2017 to December 2018. The reported trend generally declines as the number of sampled loans and exposure decline over the period. The plot is descriptive of the sampled portfolio and does not establish that time or macroeconomic changes caused the observed loss pattern.

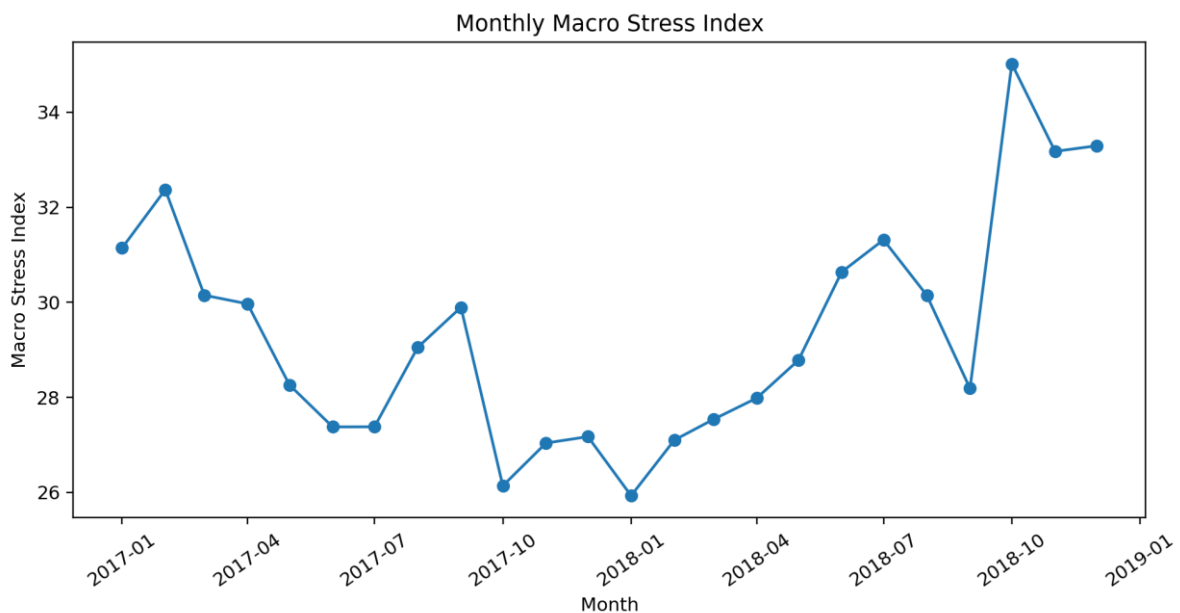


Figure 5.14. Monthly macroeconomic stress index

Figure 5.14 displays the monthly MSI over the same January 2017 to December 2018 period. The line shows that the constructed stress indicator changes from month to month rather than remaining constant. These movements provide scenario context for the portfolio analysis, but the report does not treat them as causal evidence of changes in expected loss.

The final monthly portfolio table contains 24 months from January 2017 to December 2018. Baseline expected loss declines as the number of loans and exposure decline over the sample months, while the

MSI varies across the period. Monthly values should be treated as descriptive analytics of the sampled portfolio rather than causal evidence that macroeconomic changes caused the portfolio loss pattern.

5.11 SHAP and Feature Importance

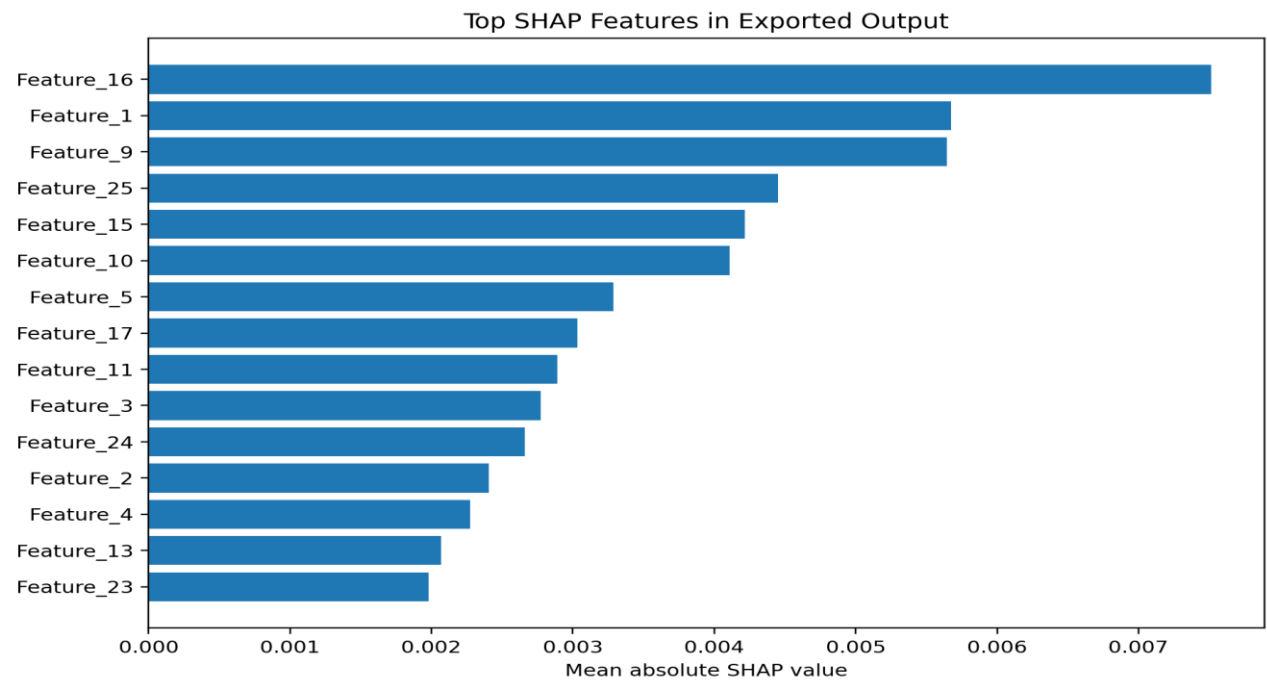


Figure 5.15A. Top exported SHAP feature importance values

Figure 5.15A presents the mean-absolute SHAP importance ranking calculated from 100 explanation samples and 26 graph features. The displayed ranking begins with Feature_16, Feature_1, Feature_9, Feature_25 and Feature_15. Because these labels are anonymized intermediate names, the ranking is useful for demonstrating the explainability output but should not be converted into named financial-feature conclusions until the mapping is verified.

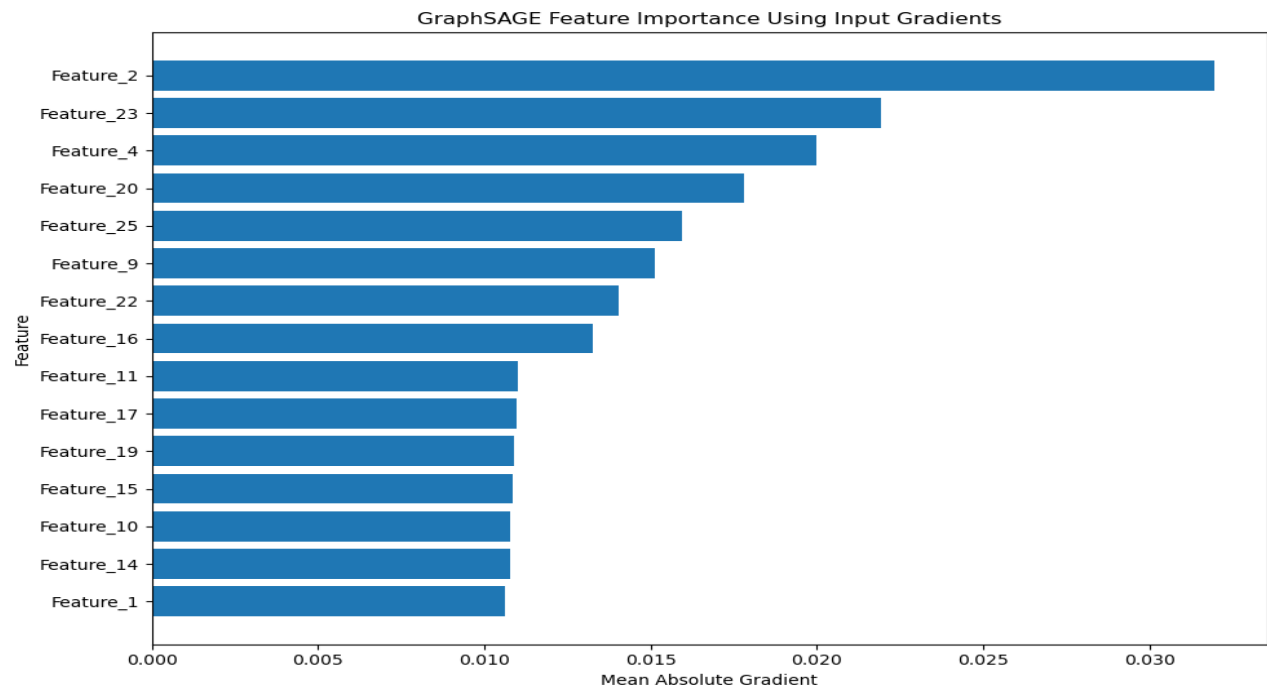


Figure 5.15B. GraphSAGE input-gradient feature importance

Figure 5.15B shows the separate GraphSAGE input-gradient importance output. Its first five displayed features are Feature_2, Feature_23, Feature_4, Feature_20 and Feature_25. Feature_25 appears in both the SHAP and gradient-based top-five lists, but this is retained only as an observed overlap because the anonymized feature-name mapping still requires verification.

The SHAP output is based on 100 explanation samples and 26 graph features in the final GNN-specific explanation stage. The mean-absolute SHAP ranking shown in the exported output begins with Feature_16, Feature_1, Feature_9, Feature_25 and Feature_15. The gradient-based GraphSAGE importance output begins with Feature_2, Feature_23, Feature_4, Feature_20 and Feature_25. The agreement of Feature_25 across these two views is interesting, but it should not be interpreted as a final scientific conclusion until the feature-name mapping is verified.

5.12 Survival Analysis

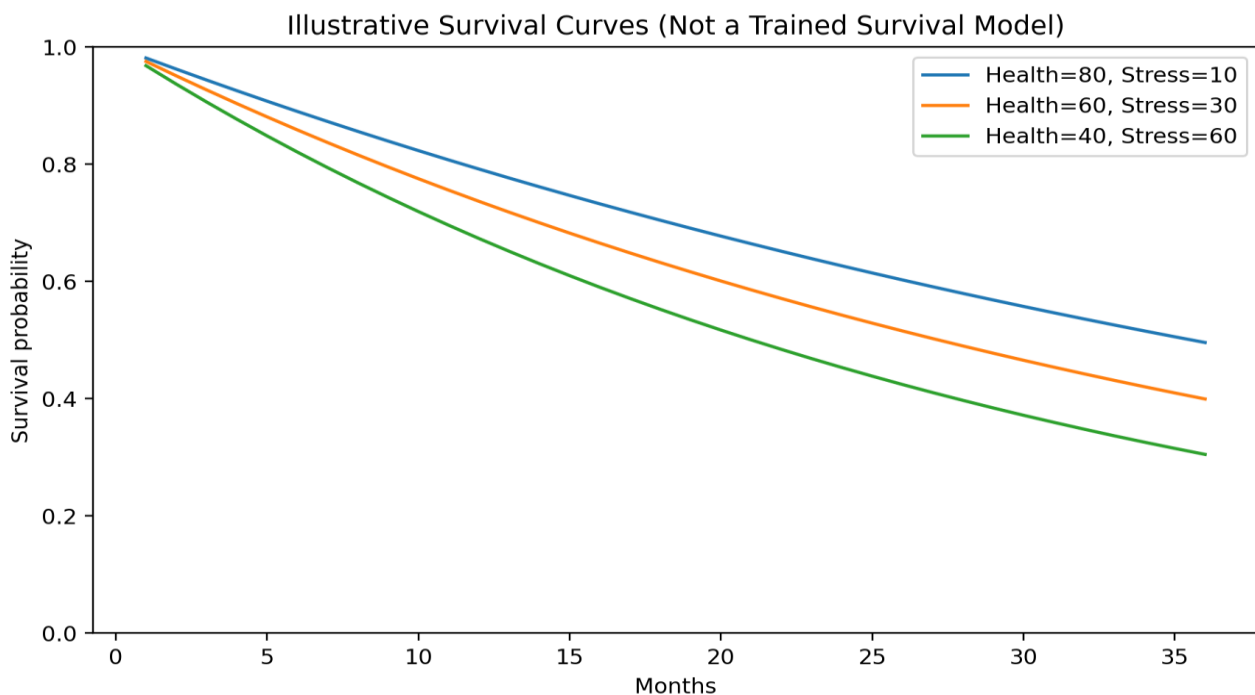


Figure 5.16. Illustrative survival curves used by the dashboard

Figure 5.16 shows how the dashboard's illustrative survival probability changes when the assumed health score and stress level are varied. The curves are generated from the project's exponential hazard formulation rather than from verified event-time observations. Therefore, the figure demonstrates scenario behaviour of the dashboard module, not an empirical estimate of time to default.

The survival curves show how the illustrative hazard changes when health score and stress level are changed. They are not empirical default-time estimates because the dataset lacks a verified event date and censoring structure. The notebook's survival-readiness check confirms that loan origination date is available but a clear observed event-date pair is not.

5.13 Methodology Audit

Table 5.7. Methodology and leakage audit

Audit item	Status	Interpretation
Target variable excluded from GraphSAGE inputs	Passed	default not present in 26 input features
Issue date excluded from GraphSAGE inputs	Passed	issue_date not used as model input
Issue year excluded from GraphSAGE inputs	Passed	issue_year not used as model input
Actual feature names recovered	Passed	original 26 names identified
Potential leakage keywords reviewed	Completed	no expected leakage flags in selected 26 features
Missing/inf values after final cleaning	Passed	zero NaN and zero infinite values reported
Graph relationship meaning	Limited	feature similarity, not verified borrower relationship
Survival analysis validity	Limited	event-time data not validated

Table 5.7 records the final methodological checks applied to the project. The target, issue date and issue year are reported as excluded from the GraphSAGE inputs; the selected 26 features have no expected leakage flags; and the final cleaned matrix has zero NaN and zero infinite values. The audit also records two important limitations: the graph represents feature similarity rather than a verified borrower relationship, and the survival analysis remains limited because event-time data were not validated.

CHAPTER 6: CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

FinTwinAI demonstrates an end-to-end research prototype for explainable financial-risk analytics. The system starts from a large loan dataset and moves through data cleaning, financial feature engineering, temporal evaluation, graph construction, GraphSAGE learning, baseline comparison, calibration, explainability, digital-twin profiling, stress testing and web-based visualization.

The most important empirical result is the difference between accuracy and default detection. The MLP achieves higher accuracy at 74.30%, but GraphSAGE achieves substantially higher default recall at 61.13%, higher F1-score at 42.19%, higher ROC-AUC at 0.6858 and higher average precision at 0.3560. This demonstrates why imbalanced credit-risk problems require several complementary evaluation measures.

The calibration experiment is another important result. Brier score falls from 0.222188 to 0.158626 for GraphSAGE and from 0.228459 to 0.167802 for MLP. The calibrated probability is then used downstream for risk categories and expected-loss calculations. This creates a more coherent analytical chain than using an uncalibrated class score directly.

The digital-twin layer adds a portfolio interpretation. For the 5,000 final profiles, total exposure is 72,858,850 and baseline expected loss is 35,861,092.02. Under the project's hypothetical stress assumptions, expected loss rises to 42,763,119.00 under moderate stress and 51,566,807.22 under severe stress.

6.2 Main Contributions

- An integrated research workflow combining tabular and graph neural modelling.
- A KNN feature-similarity graph with 30,000 nodes and 369,098 final edge entries.
- Direct comparison between a 6,018-parameter MLP and an 11,906-parameter GraphSAGE model.
- Probability calibration evaluated with Brier score and reliability diagrams.
- A digital-twin profile that links PD, health score, exposure, expected loss and macro stress.
- Portfolio-level and scenario-level risk aggregation.
- An interactive Streamlit dashboard designed for simple interpretation.
- An explicit methodology audit and limitation framework for responsible research reporting.

The study also has data and validation limitations, including a 40-row discrepancy between reported modelling counts, anonymized intermediate SHAP feature labels, and evaluation on one sampled temporal test set without external validation or confidence intervals. Exposure and expected-loss values are therefore reported in the original dataset units and should not be treated as certified lending or financial-advice outputs.

The macroeconomic and survival components are research assumptions rather than fully validated forecasting models. The stress multipliers are hypothetical, the MSI is a constructed index, and the survival module lacks verified event dates and censoring information, so these outputs are mainly useful for scenario-based and educational analysis.

The KNN graph represents feature similarity rather than a verified borrower-to-borrower or financial-contagion network, and the GraphSAGE experiment uses a 30,000-node sample. Therefore, the graph results should be interpreted as contextual learning from similar financial profiles rather than evidence of real network relationships.

6.3 Key Limitations

The system is not a certified credit score, lending decision engine or financial-advice system.

A practical next step is to develop FinTwinAI into a governed financial-risk analytics platform that can work with regularly updated portfolio data. This would include authenticated access, automated data-quality and drift monitoring, model-version tracking and controlled dashboard deployment. The graph could also be rebuilt from verified and ethically collected financial relationships where such information is available. These additions would make the prototype more suitable for controlled industry research and portfolio-monitoring workflows while preserving responsible-use safeguards.

A further research direction is to strengthen the analytical models through richer longitudinal data and broader validation. Future experiments can use verified event-time data for survival modelling, exact feature-name mapping for explainability, nested temporal validation and additional calibration measures. The macroeconomic module can also be extended with validated scenario narratives and lagged economic variables. Such work would help examine the robustness and generalizability of the integrated FinTwinAI framework across different datasets and financial conditions.

6.4 Future Work

Future development can focus on transforming FinTwinAI into a more practical and reliable financial-risk analytics platform. The system can be extended with regularly updated portfolio data, authenticated user access, automated data-quality and model-drift monitoring, and model-version tracking. Verified borrower relationships or transaction-based networks can also be incorporated instead of relying only on feature-similarity graphs. These improvements would support controlled deployment for portfolio monitoring and risk assessment while maintaining appropriate governance, privacy, and responsible-use safeguards.

Further research can improve the analytical components through richer longitudinal datasets and validation across multiple financial datasets and market conditions. Survival analysis can be strengthened using verified event dates, follow-up periods, and censoring information, while the exact mapping of graph features to their original financial meanings can improve explainability. Future experiments can also use nested temporal validation, additional calibration measures, and more robust macroeconomic scenarios with lagged economic variables. These extensions would help evaluate the robustness, stability, and generalizability of FinTwinAI beyond the current experimental setting.

6.5 Final Statement

FinTwinAI is best understood as an integrated academic research prototype that demonstrates how modern machine-learning methods can be connected with established financial-risk concepts and presented through a practical web-based interface. The project brings together data preprocessing, financial feature engineering, temporal evaluation, KNN-based feature-similarity graph construction, GraphSAGE learning, baseline MLP comparison, probability calibration, SHAP-based explainability, digital-twin profiling, portfolio expected-loss analysis, macroeconomic stress scenarios and Streamlit visualization. The use of GraphSAGE provides an inductive neighbourhood-aggregation framework [1], while SHAP supports feature-level interpretation of model predictions [2] and probability calibration provides a more meaningful basis for downstream risk analysis [3]. The final dashboard therefore demonstrates an end-to-end analytical workflow rather than a single model result. At the same time, the system should remain a research and educational prototype until the data, labels, calibration, explainability, survival analysis and macroeconomic assumptions are independently validated. In particular, the reported exposure, expected-loss and stress outputs should not be

interpreted as certified lending decisions or financial advice. Overall, the project provides a structured foundation that can be further tested, reproduced and extended with stronger longitudinal data, broader validation, verified financial relationships and appropriate governance before any operational use.

REFERENCES

- [1] W. L. Hamilton, R. Ying and J. Leskovec, “Inductive Representation Learning on Large Graphs,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 1024–1034, 2017.
<https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e-Abstract.html>
- [2] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 4765–4774, 2017.
<https://papers.nips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf>
- [3] J. C. Platt, “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods,” in Advances in Large Margin Classifiers, MIT Press, 1999.
<https://www.microsoft.com/en-us/research/project/support-vector-machines/publications/>
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [5] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Advances in Neural Information Processing Systems, vol. 32, 2019.
https://papers.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html
- [6] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
<https://arxiv.org/abs/1903.02428>
- [7] D. R. Cox, “Regression Models and Life-Tables,” Journal of the Royal Statistical Society: Series B (Methodological), vol. 34, no. 2, pp. 187–202, 1972. <https://doi.org/10.1111/j.2517-6161.1972.tb00899.x>
- [8] C. Croux, J. Jagtiani, T. Korivi and M. Vulcanovic, “Important Factors Determining Fintech Loan Default: Evidence from the LendingClub Consumer Platform,” Journal of Economic Behavior & Organization, vol. 173, pp. 270–296, 2020, doi: 10.1016/j.jebo.2020.03.016.
<https://doi.org/10.1016/j.jebo.2020.03.016>
- [9] J. D. Turiel and T. Aste, “Peer-to-peer loan acceptance and default prediction with artificial intelligence,” Royal Society Open Science, vol. 7, no. 6, Art. 191649, 2020, doi: 10.1098/rsos.191649.
<https://doi.org/10.1098/rsos.191649>
- [10] J. R. Bhat, S. A. AlQahtani and M. Nekovee, “FinTech enablers, use cases, and role of future internet of things,” Journal of King Saud University – Computer and Information Sciences, vol. 35, no. 1, pp. 87–101, 2023, doi: 10.1016/j.jksuci.2022.08.033.
<https://doi.org/10.1016/j.jksuci.2022.08.033>
- [11] P. Chatterjee, D. Das and D. B. Rawat, “Digital twin for credit card fraud detection: Opportunities, challenges, and fraud detection advancements,” Future Generation Computer Systems, vol. 158, pp. 410–426, 2024, doi: 10.1016/j.future.2024.04.057.
<https://doi.org/10.1016/j.future.2024.04.057>
- [12] Streamlit, “Streamlit Documentation: App Development and API Reference,” accessed September 2026. <https://docs.streamlit.io/>
- [13] SHAP, “SHAP Documentation: SHapley Additive exPlanations,” accessed September 2026.
<https://shap.readthedocs.io/en/latest/>

[14] Federal Reserve Bank of St. Louis, FRED, “Federal Reserve Economic Data,” macroeconomic data source used in the project, accessed September 2026. <https://fred.stlouisfed.org/>

[15] LendingClub-derived accepted loan dataset, used as the source dataset in the project notebook. The project uses the accepted-loan data distributed through the project source; the exact raw-file version and download date should be preserved in the final submission package for reproducibility. <https://www.kaggle.com/datasets/wordsofthewise/lending-club>

ANNEXURES

Annexure A: Core Mathematical Formulae

Table A.1. Core mathematical formulae

Quantity	Formula
Probability of default	$P_i = P(Y_i=1 X_i)$
Health score	Health Score = $(1-P_i) \times 100$
Risk category	Low <0.20 ; Medium $0.20- <0.40$; High $0.40- <0.60$; Very High ≥ 0.60
Euclidean distance	$d(x_i, x_j) = \sqrt{\sum_m (x_{im} - x_{jm})^2}$
Graph initial edges	$N \times k = 30,000 \times 8 = 240,000$
GraphSAGE layer	$h_v^{(k)} = \sigma(W_k \cdot \text{CONCAT}(h_v^{(k-1)}, \text{MEAN}(\{h_u^{(k-1)} : u \in N(v)\})))$
Brier score	Brier = $(1/n) \sum_i (P_i - y_i)^2$
Expected loss	$EL_i = \text{Exposure}_i \times PD_i$
Moderate stress PD	$\min(1, 1.20 \times PD_{\text{baseline}})$
Severe stress PD	$\min(1, 1.50 \times PD_{\text{baseline}})$
EMI	$P \cdot r[(1+r)^n / (1+r)^{n-1}]$
Loan-to-income	Loan Amount / Annual Income
Installment-to-income	$(12 \times \text{Installment}) / \text{Annual Income}$
Illustrative survival	$S(t) = e^{(-\lambda t)}$
Illustrative hazard	$\lambda = 0.015 \times [1 + (100 - \text{Health Score})/100 + (\text{Stress}/100)]$

Annexure B: Full Monthly Portfolio Table

Table B.1. Full monthly portfolio table

Month	Loans	Exposure	Avg PD	Baseline EL	Moderate EL	Severe EL	MSI
2017-01	357	5,208,500	0.457332	2,546,967	3,046,602	3,684,668	31.141166
2017-02	325	5,028,850	0.479525	2,568,745	3,073,415	3,687,151	32.368806
2017-03	390	5,688,000	0.446837	2,675,243	3,192,533	3,879,080	30.148960
2017-04	296	4,302,675	0.469082	2,116,483	2,518,582	3,052,284	29.966869
2017-05	327	4,314,525	0.470029	2,180,582	2,602,204	3,151,588	28.262410
2017-06	390	5,489,450	0.469692	2,747,261	3,290,848	3,975,442	27.378801
2017-07	332	4,823,875	0.477747	2,388,814	2,847,989	3,420,477	27.380581
2017-08	354	4,970,950	0.459485	2,459,028	2,939,226	3,555,143	29.052068
2017-09	267	4,103,725	0.497994	2,178,967	2,589,734	3,069,872	29.894749
2017-10	258	3,562,600	0.460028	1,793,445	2,136,316	2,581,592	26.140634
2017-11	262	3,967,625	0.451397	1,926,123	2,300,865	2,769,760	27.039540

2017-12	235	3,343,525	0.468957	1,737,917	2,067,456	2,463,284	27.173398
2018-01	177	2,591,675	0.439730	1,225,081	1,457,707	1,761,487	25.931780
2018-02	149	2,142,725	0.439749	998,043	1,195,994	1,439,111	27.101689
2018-03	151	2,394,525	0.423912	1,095,426	1,302,618	1,573,507	27.536452
2018-04	171	2,574,300	0.434672	1,231,349	1,454,725	1,741,563	27.983738
2018-05	131	1,978,825	0.443498	969,283	1,151,603	1,399,188	28.782340
2018-06	116	1,826,025	0.460587	942,536	1,116,977	1,340,161	30.638570
2018-07	80	1,315,850	0.433618	620,641	738,234	905,616	31.314138
2018-08	76	937,700	0.421276	404,170	485,004	599,919	30.148290
2018-09	48	623,850	0.409312	261,224	313,469	387,052	28.187983
2018-10	44	746,500	0.464007	388,589	454,809	538,935	35.015764
2018-11	35	529,575	0.429405	254,076	304,891	364,610	33.174277
2018-12	29	393,000	0.377136	151,101	181,321	225,316	33.292840

Annexure C: Dashboard Page Inventory

Table C.1. Dashboard page inventory

Page	Key displayed content
Executive Dashboard	5,000 records; 26 features; 30,000 graph nodes; 369,098 graph edges; model comparison; calibration; pipeline
Personal Financial Health	Income, loan, interest, term, EMI, expenses, credit score, utilization, delinquencies, health score
Portfolio Analytics	Borrower count, exposure, expected loss and risk-category distribution
GraphSAGE Network Risk	Graph statistics, confusion matrix, model metrics and network limitations
SHAP Explainability	SHAP importance, explanation purpose and feature contribution
Model Performance	Accuracy, precision, recall, F1, ROC-AUC, average precision, calibration and training settings
Macro Stress Testing	Baseline/moderate/severe stress, macro indicators and limitations
Survival Analysis	Illustrative survival probability and scenario controls
About Project	Student, supervisor, methods and academic-use disclaimer

Annexure D: Reproducibility Checklist

- Preserve the exact raw dataset file and download date.
- Record Python, pandas, scikit-learn, PyTorch, PyTorch Geometric and SHAP versions.
- Save train, validation and test node indices.
- Save the scaler and training-node median values.
- Save the exact 26-feature list and its mapping to SHAP outputs.
- Save the 30,000-node graph sample and edge index.
- Save the GraphSAGE and MLP model weights.

- Save the selected threshold and calibration model.
- Save reliability-curve data and Brier scores.
- Save macroeconomic source series and MSI construction code.
- Save the exact stress multipliers.
- Save dashboard CSV schemas and output files.
- Run the leakage and missing-value audits before each final model run.
- Record any changes made to the dashboard source code after model training.

Annexure E: Key Numerical Results at a Glance

Table E.1. Key numerical results at a glance

Indicator	Final value
Complete dataset records	2,260,701
Reported binary usable records	1,345,310
Reported modelling-table records	1,345,350
Non-default records	1,076,751
Default records	268,599
Graph nodes	30,000
Graph features	26
Final graph edge entries	369,098
GraphSAGE parameters	11,906
MLP parameters	6,018
GraphSAGE accuracy	64.32%
GraphSAGE recall	61.13%
GraphSAGE F1	42.19%
GraphSAGE ROC-AUC	0.6858
MLP accuracy	74.30%
MLP recall	9.77%
MLP ROC-AUC	0.5533
GraphSAGE Brier before calibration	0.222188
GraphSAGE Brier after calibration	0.158626
Dashboard profiles	5,000
Total exposure	72,858,850
Baseline expected loss	35,861,092.02
Moderate-stress expected loss	42,763,119.00
Severe-stress expected loss	51,566,807.22

Annexure F: Methodology Interpretation Notes

The report deliberately distinguishes between verified outputs and research assumptions. The GraphSAGE metrics, confusion matrix, graph statistics, calibration values and portfolio totals are taken from the supplied project workflow. The KNN graph is interpreted as feature similarity. The macro stress multipliers are explicitly hypothetical. The survival curves are illustrative. The digital-twin term describes an analytical profile. These distinctions are included so that the project can be presented as a serious academic study without overstating what the data can prove.